

Digital Design

Latch and FF

with Verilog Model

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

**Some part of Slides Adopted from Digital System Book
by Frank Vahid**

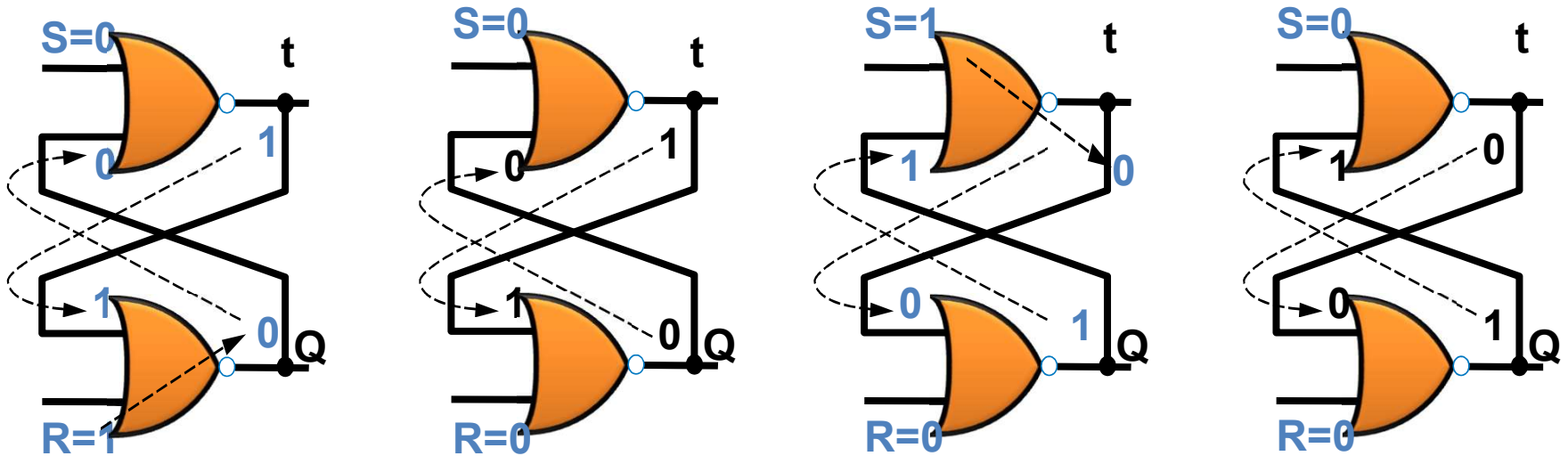
Announcement

- CS242 System Software Lab (others may ignore)
 - Deferred Written Exam scheduled Today at 3PM
 - Room 2201: 231010101 – 2310101065
 - Room 2202: rest of CS242 students
- No CS221 Class on Post Puja week
 - Oct 14 to Oct 16
- Makeup class on
 - 27th Sept Friday 8-9AM-5G2
 - 04 Oct Friday 8-9AM-5G2

Outline

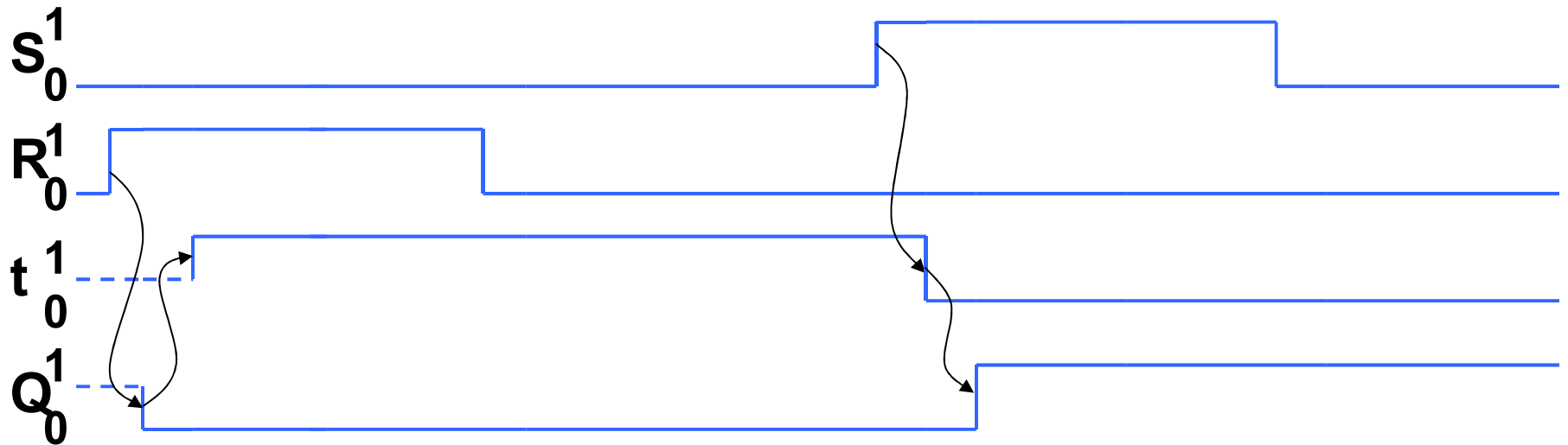
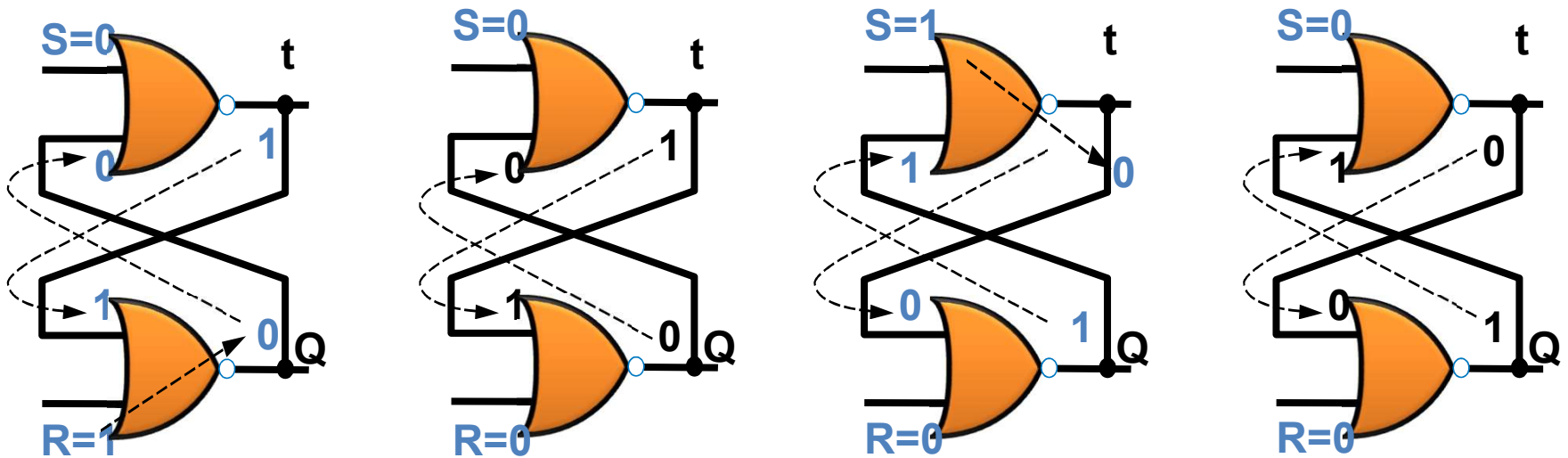
- Combinational Vs Sequential Logic Design
- Design a **Circuit**, that stores one bit
 - RS latch
- Stabilizing RS latch : Level Sensitive
- Clocked Latch : Flip Flop- Edge Sensitive
- D, JK, T flip flops
- Characterization Table and Equation
 - RS, D, JK and T Flip flop

Bit Storage Using an SR Latch

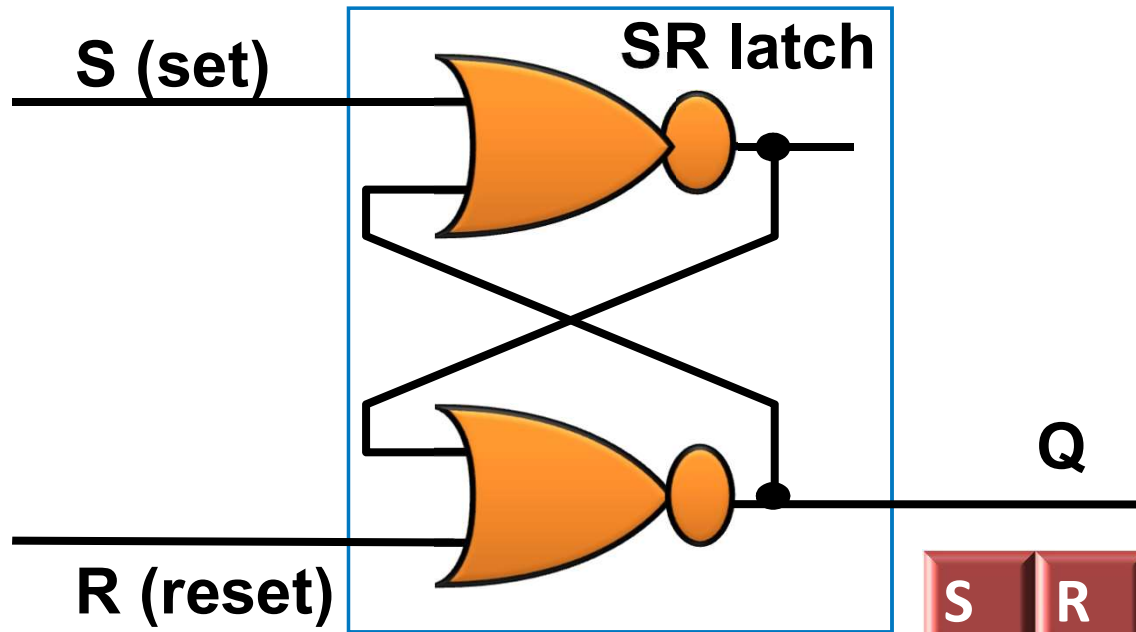


- When $R=1$, $S=0 \rightarrow$ As $R=1$, ensure Q set to 0, As $S=0$ and $Q=0$ it result $t=1$
- When $R=0$, $S=0 \rightarrow$ As $t=1$, Q set to 0, As $S=0$ and $Q=0$ it result $t=1$ (State Continue)
- When $R=0$, $S=1 \rightarrow$ As $S=1$, ensure t set to 0 and $Q=1$
- When $R=0$, $S=0 \rightarrow$ As $t=0$, Q set to 1, As $S=0$ and $Q=1$ it result $t=0$ (State Continue)

Bit Storage Using an SR Latch



Function Table of SR Latch



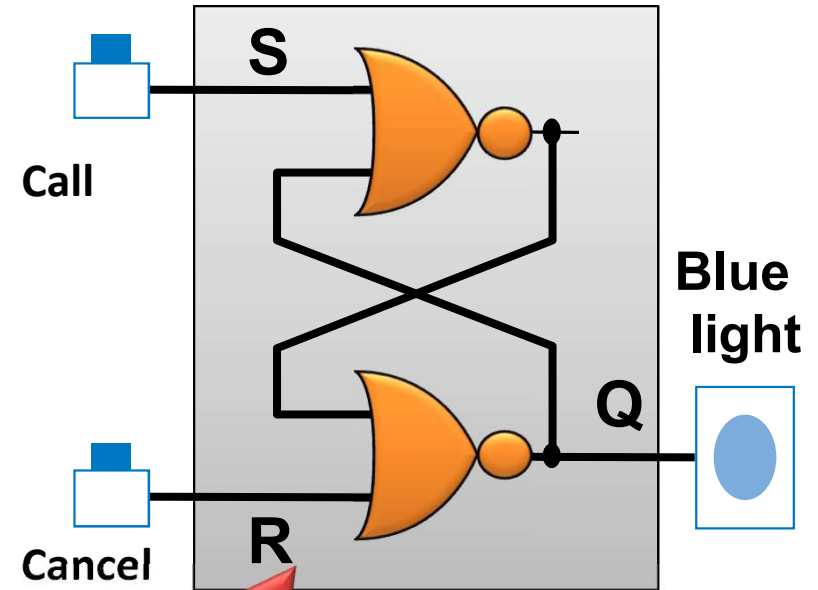
S	R	Q	Q'	
1	0	1	0	
0	0	1	0	After S=1, R=0
0	1	0	1	
0	0	0	1	After S=0, R=1
1	1	0	0	Forbidden

Set : make out put 1

Reset : make out put 0

Problem with SR Latch: $SR=11$

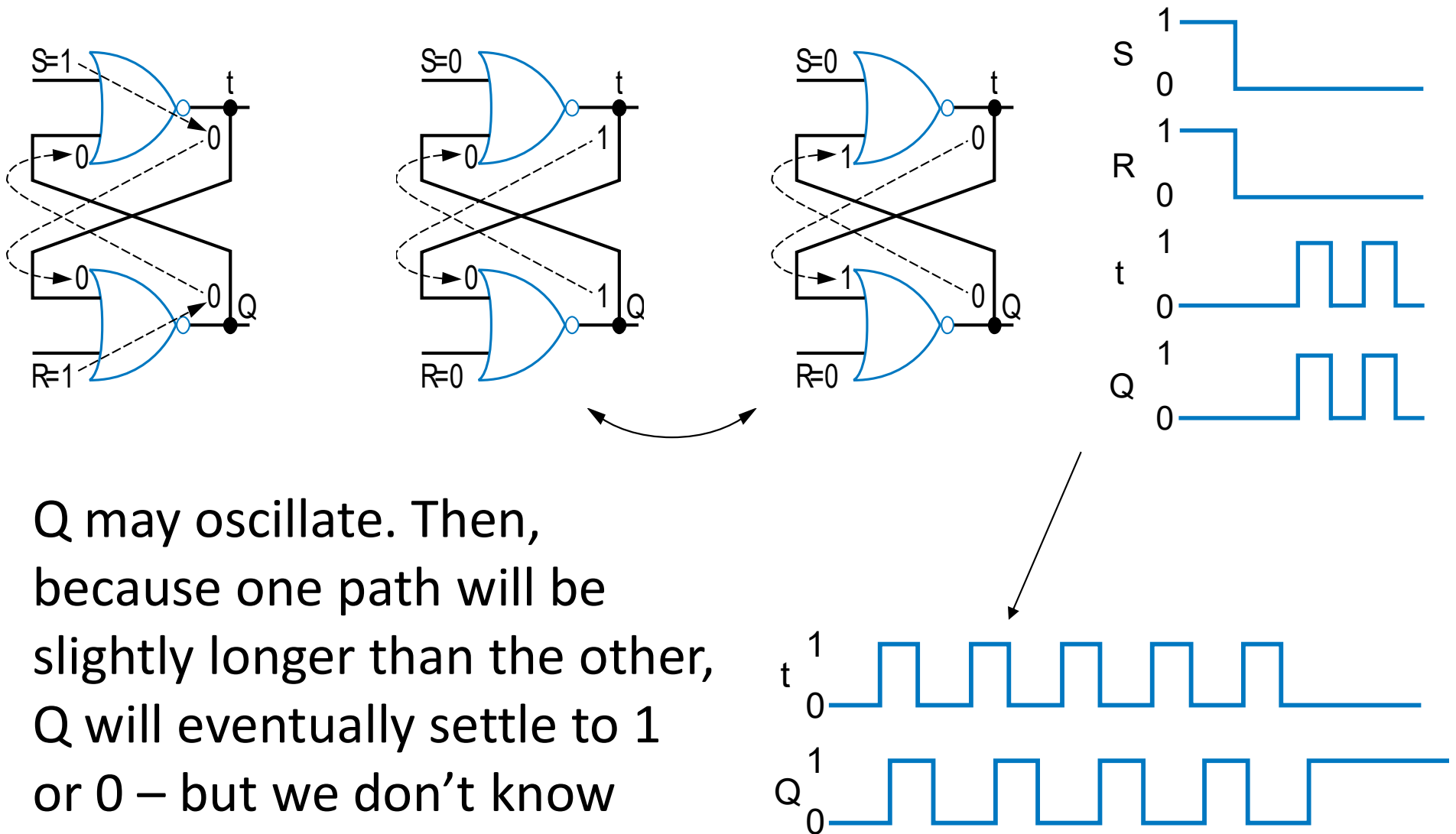
- Problem
 - If $S=1$ and $R=1$ simultaneously, we don't know what value Q will take (Unpredictable)



Race Condition :-

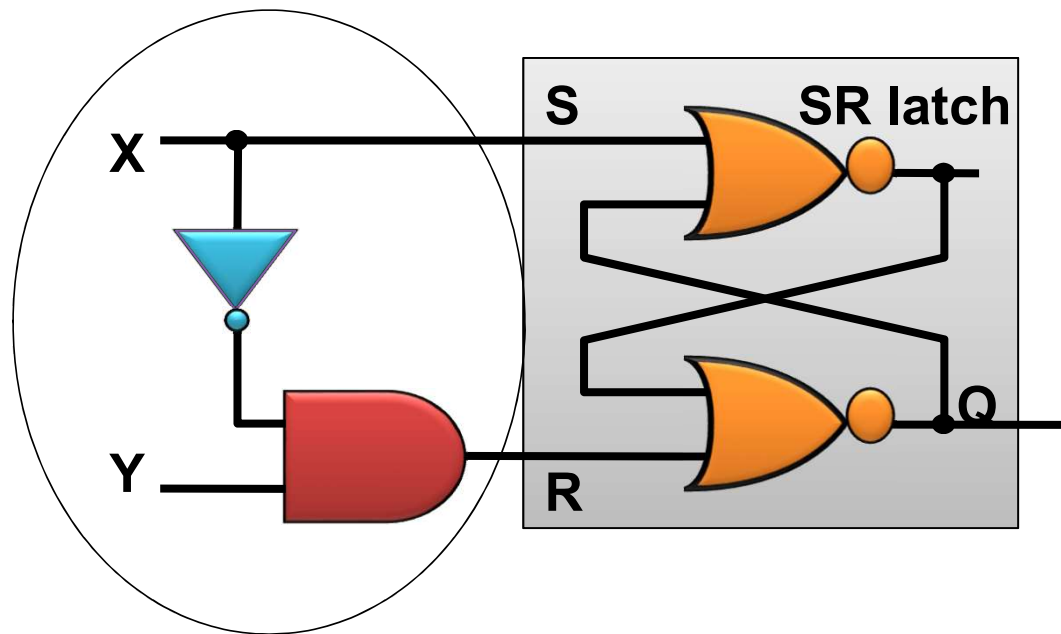
**Who will win 0 or 1?
Eventually, what will be the
value of Q ?**

Problem with SR Latch



Solution to Race Condition

- We try to avoid $S=1$ and $R=1$ by the following circuit



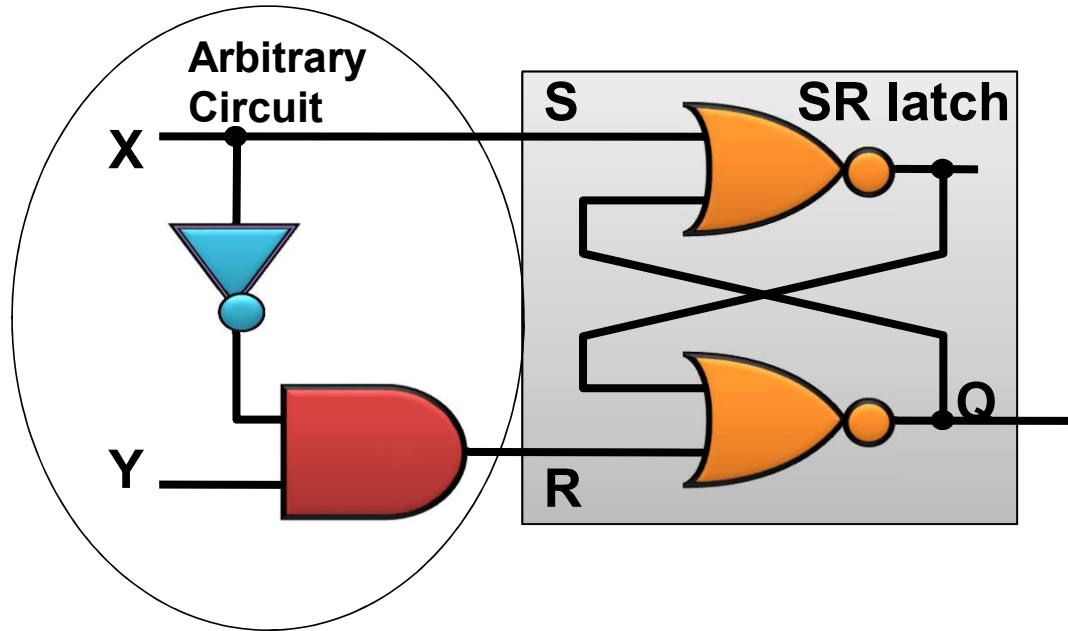
- Can we ensure value $S=1$ and $R=1$ will not happen at the same time?

Problem with SR Latch

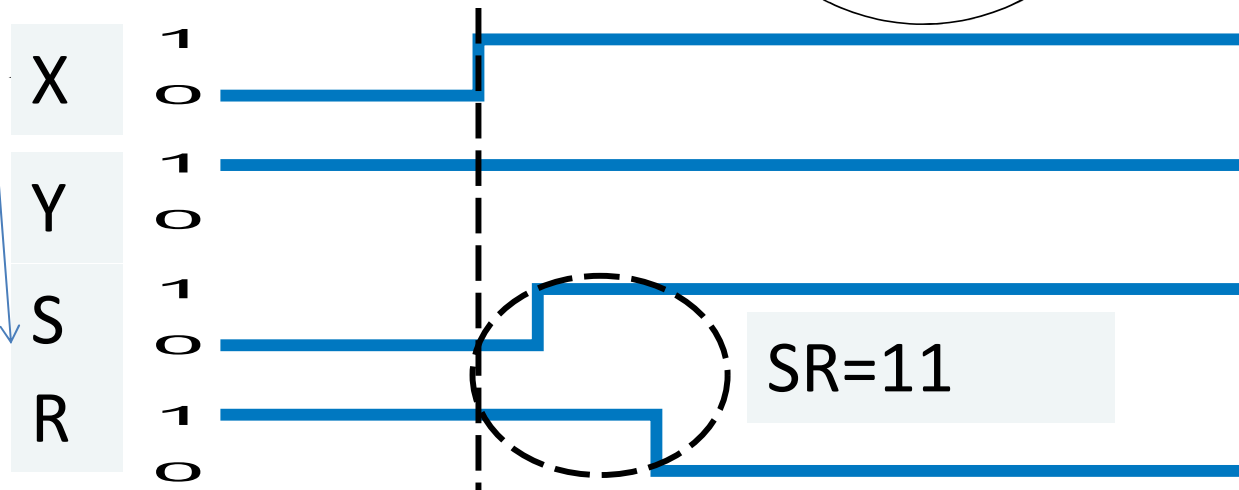
- Problem not just one of a user pressing two buttons at same time
- Can also occur even if SR inputs come from a circuit that supposedly never sets $S=1$ and $R=1$ at same time
 - But does, due to different delays of different paths

Problem with SR Latch

- $X=1$ and $Y=1$ set



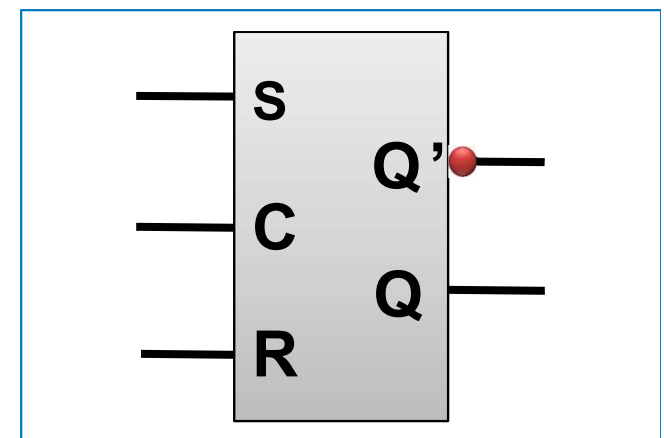
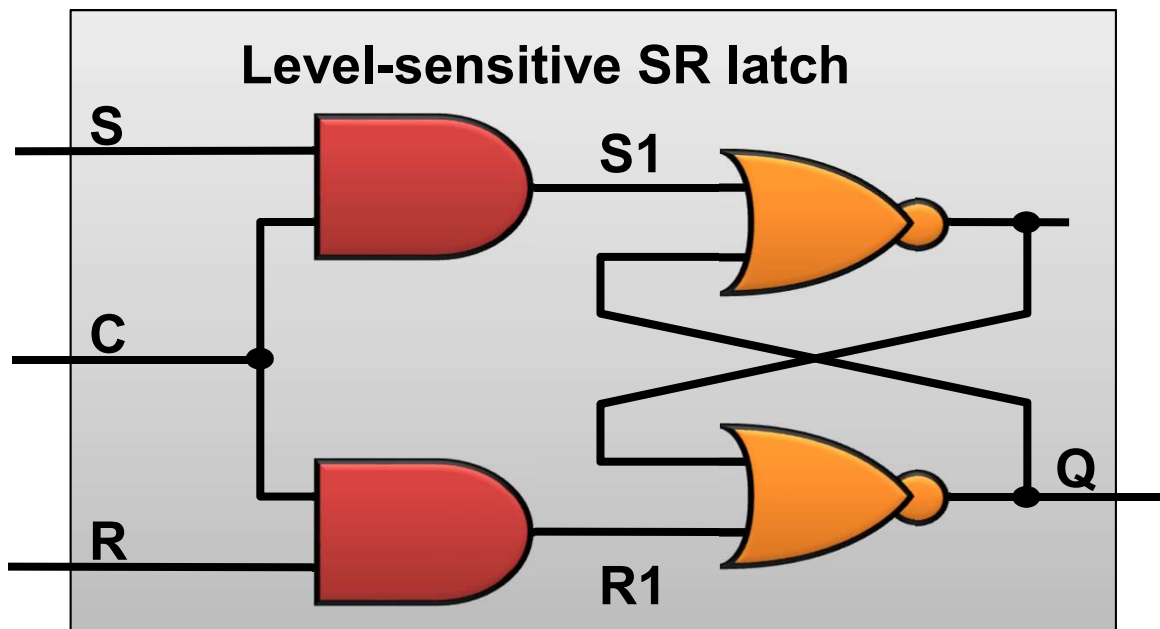
Yesterday it was
my mistake



The longer path from X to R than to S causes $SR=11$ for short time – could be long enough to cause oscillation

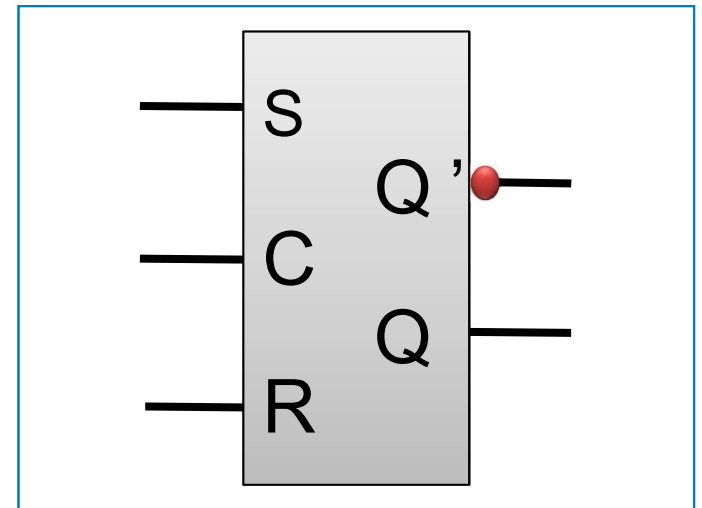
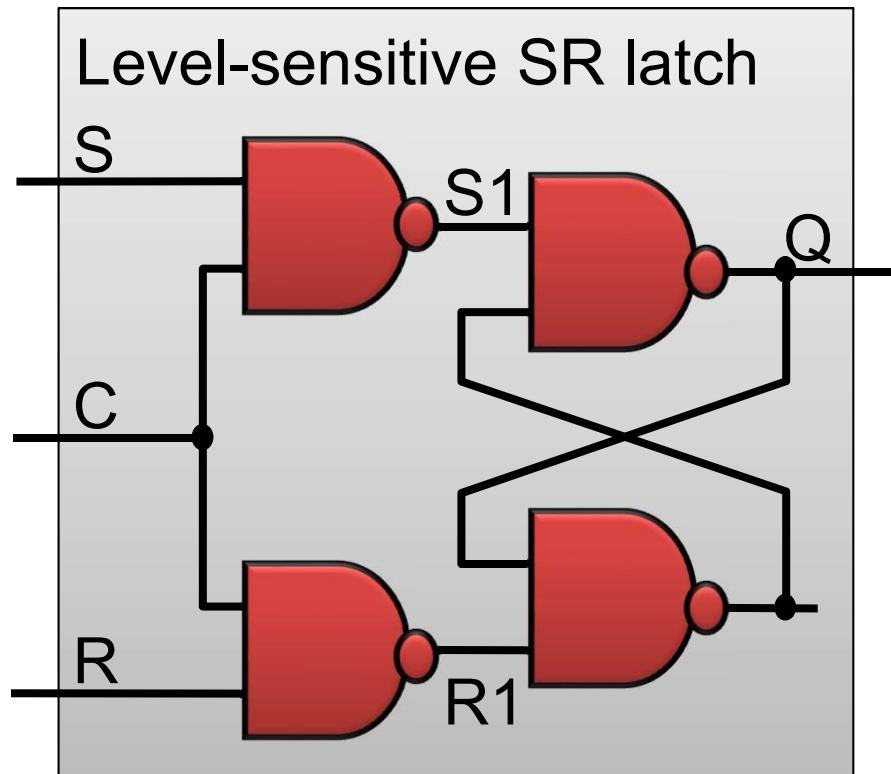
Solution: Level-Sensitive SR Latch

- Add enable input “C or En”
 - Only let S and R change when $C = 0$: Ensure circuit in front of SR never sets $SR = 11$, except briefly due to path delays
 - Change C to 1 only after sufficient time for S and R to be stable
 - When C becomes 1, the stable S and R value passes through the two AND gates to the SR latch’s S1 R1 inputs.



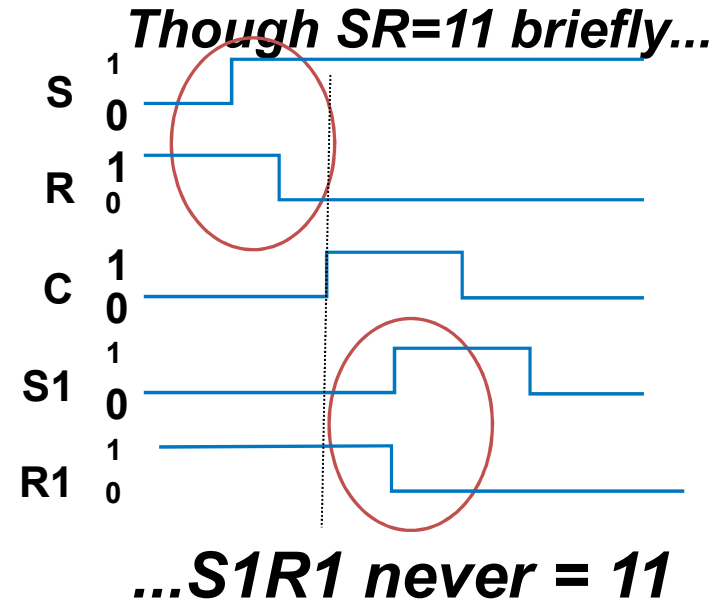
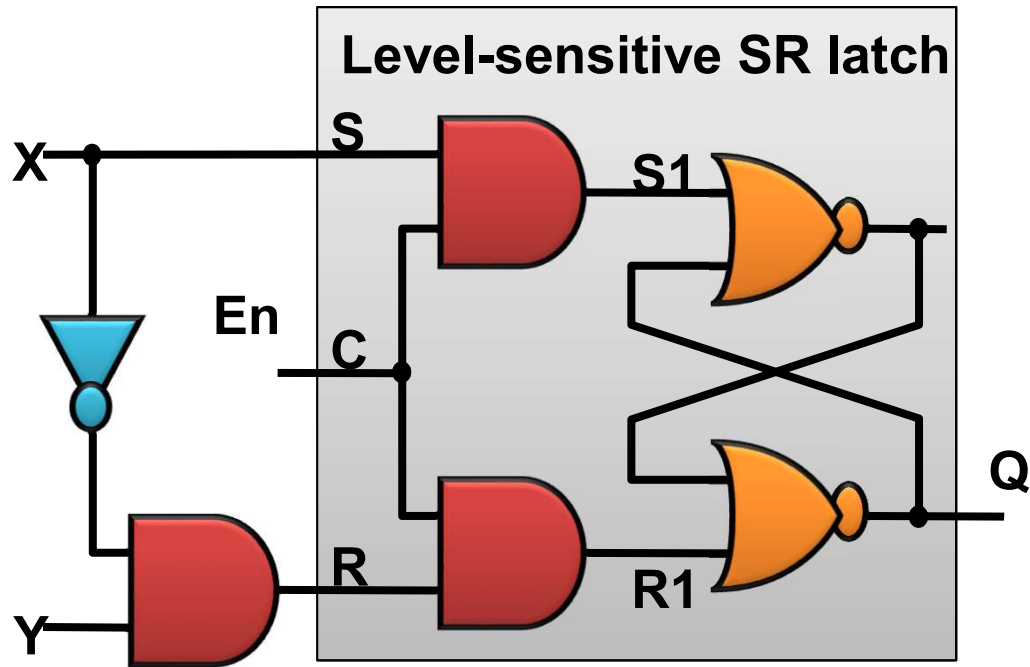
Level-sensitive SR latch symbol

Level Sensitive: SR Latch with NAND Gate

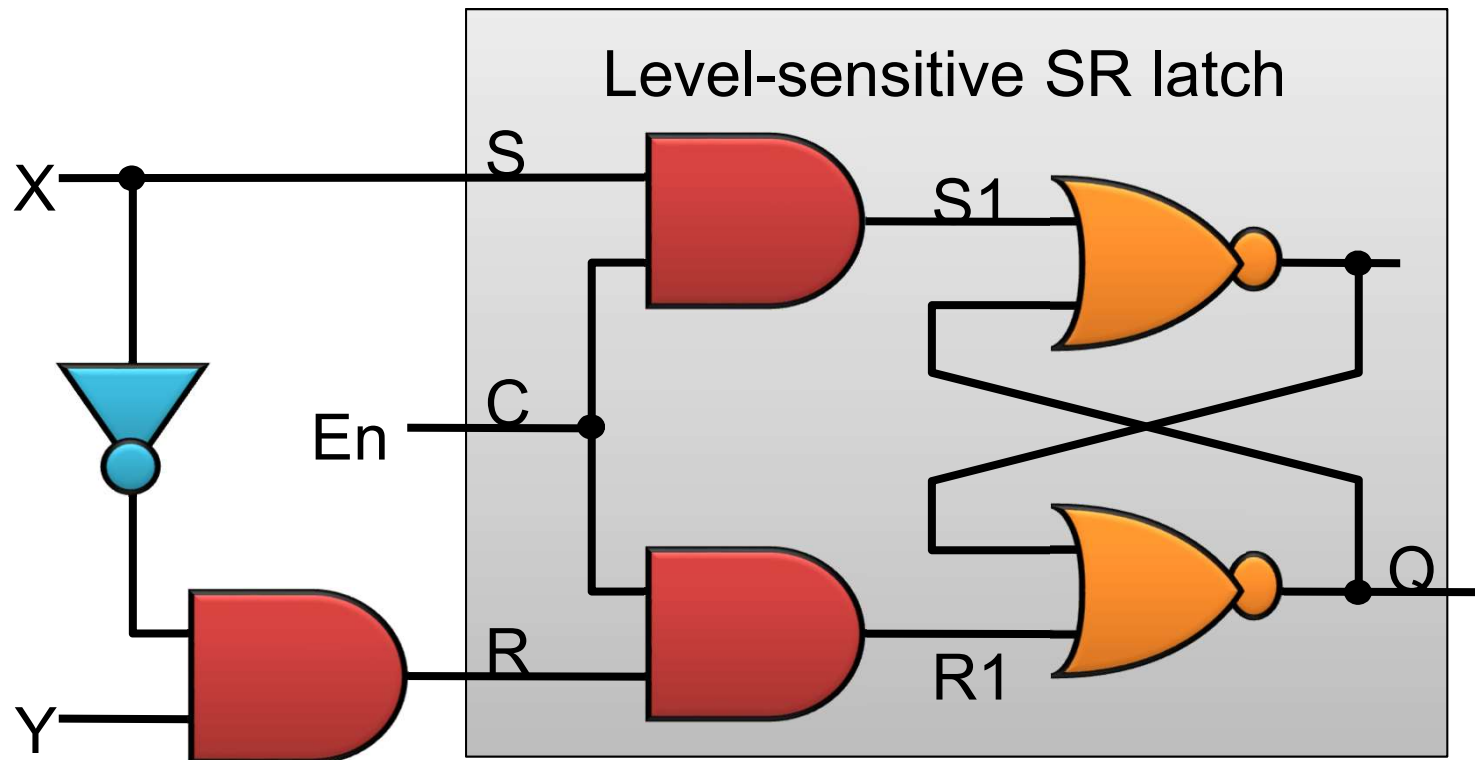


Level-sensitive SR latch symbol

Ensure $S=1$ and $R=1$ should not happened to Level-Sensitive SR Latch



Solution: Ensure, Stabilize, Store



Ensure
Stage

Never Happens
SR=11

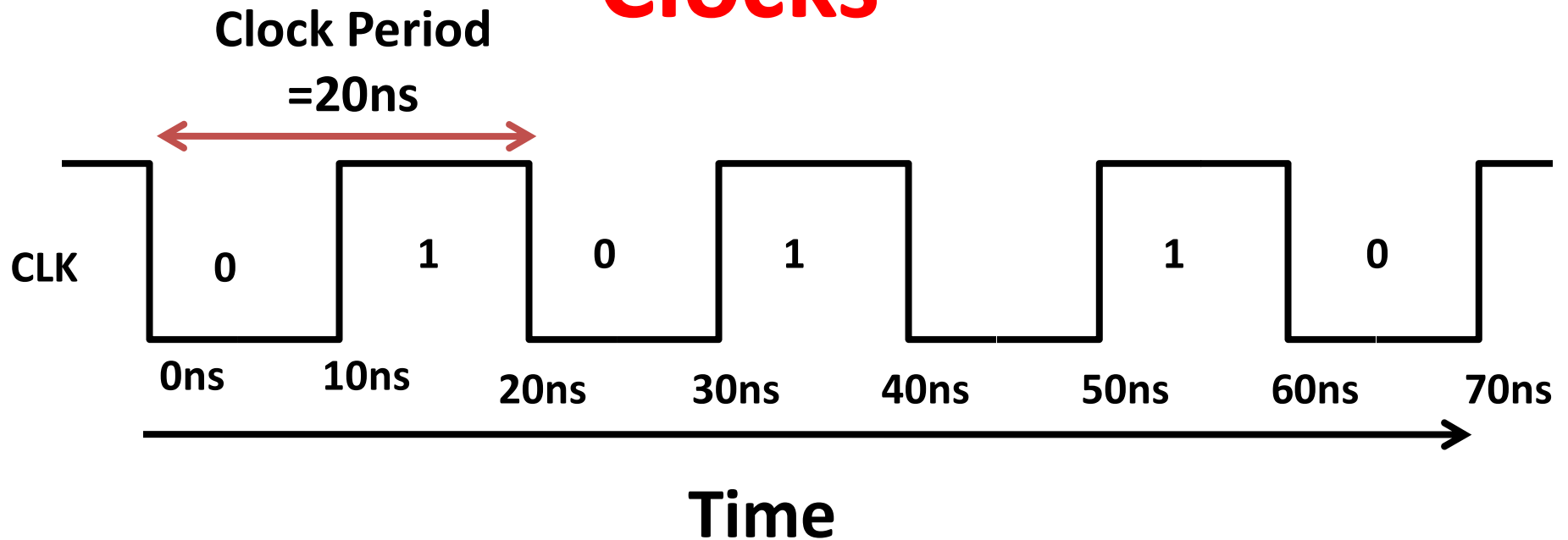
Stabilize
Stage

When C=0
Stabilize SR and
Use when C=1

Store
Stage

Store bit

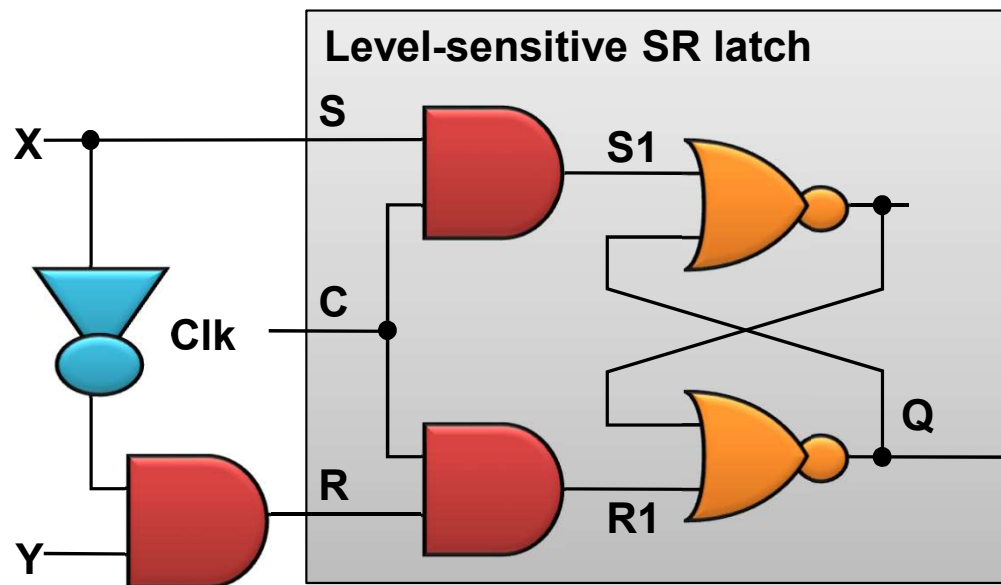
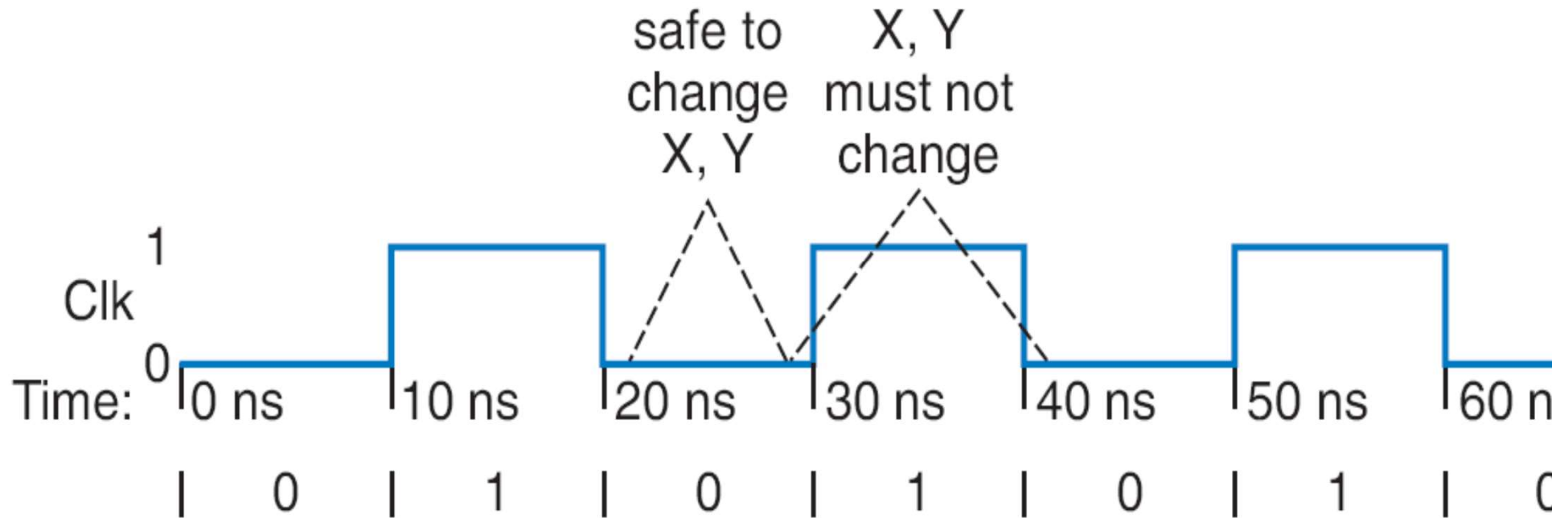
Clocks



- ***Clock period***: time interval between pulses
- ***Clock cycle***: one such time interval
- ***Clock frequency***: $1/\text{period}$
 - frequency = $1 / 20 \text{ ns} = 50 \text{ MHz}$
 - $1 \text{ Hz} = 1/\text{s}$

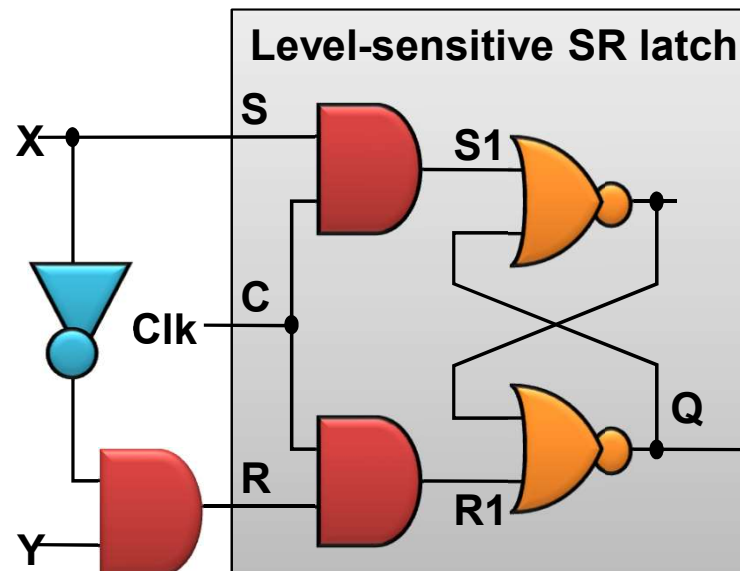
Freq	Period
1 GHz	1 ns
100 MHz	10 ns
10 MHz	100 ns

Clock Signal for RS latch



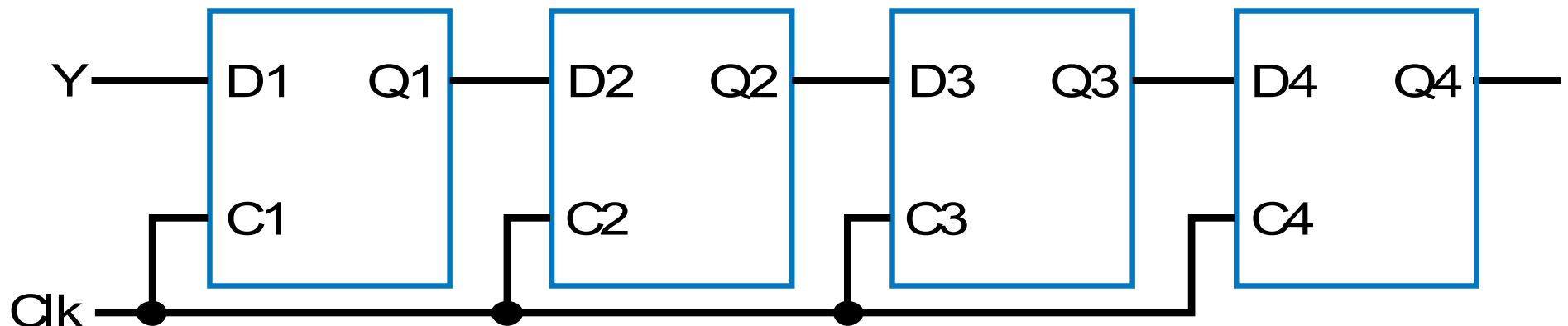
Clock Signals for a RS Latch

- How do we know when it's safe to set $C=1$?
 - Most common solution – make C pulse up/down
 - $C=0$: Safe to change X, Y $C=1$: Must *not* change X, Y
 - **Clock** signal -- Pulsing signal used to enable latches
 - Because it ticks like a clock
 - Sequential circuit whose storage components all use clock signals: **synchronous** circuit



Level-Sensitive D Latches

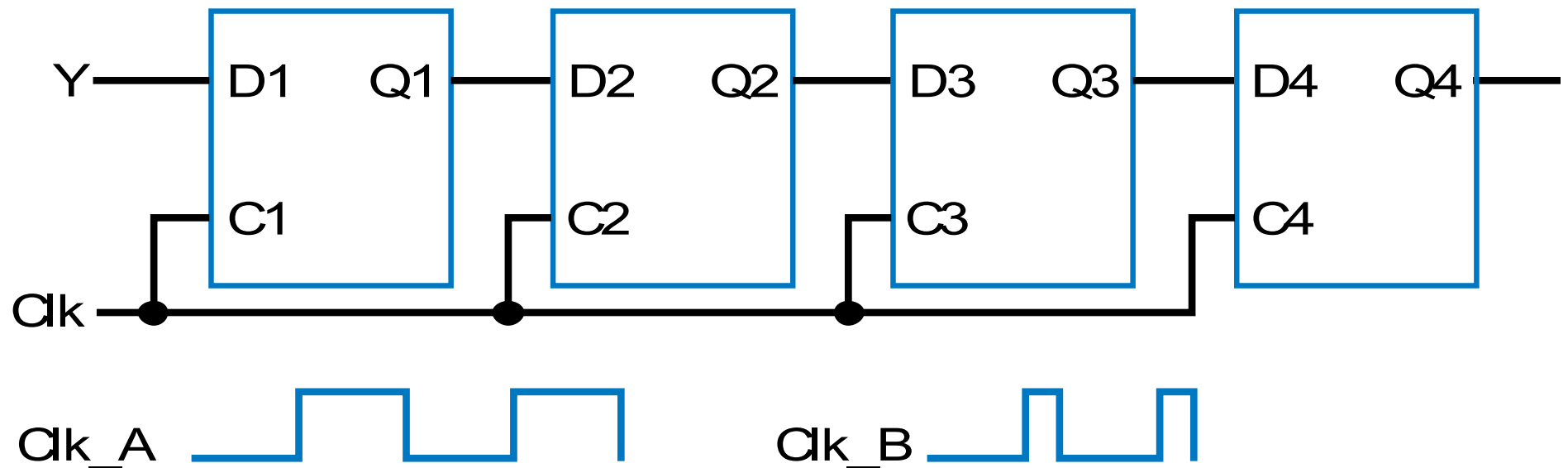
- Suppose D FFs are arranged in linear fashion connected using a single clock signal Clk
- Every clock we want to shift one bit to right
 - Right shift one bit per cycle



**Does this circuit (with level sensitive D latch)
Shift one bit per cycle ?**

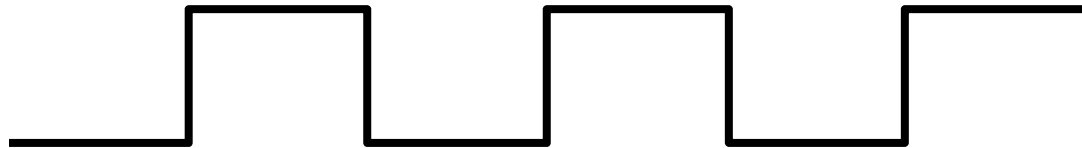
Problem with Level-Sensitive D Latch

- D latch still has problem (as does SR latch)
 - When $C=1$, through how many latches will a signal travel?
 - Depends on for how long $C=1$
 - Clk_A -- signal may travel through multiple latches
 - Clk_B -- signal may travel through fewer latches
- Hard to pick C that is just the right length**



Problem with Level-Sensitive D Latch

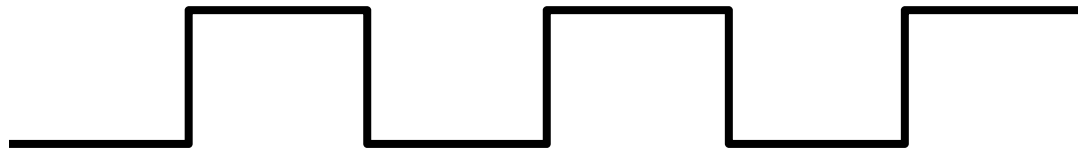
- We want do the work: one per clock cycle
 - Independent of length of clock (1 time)



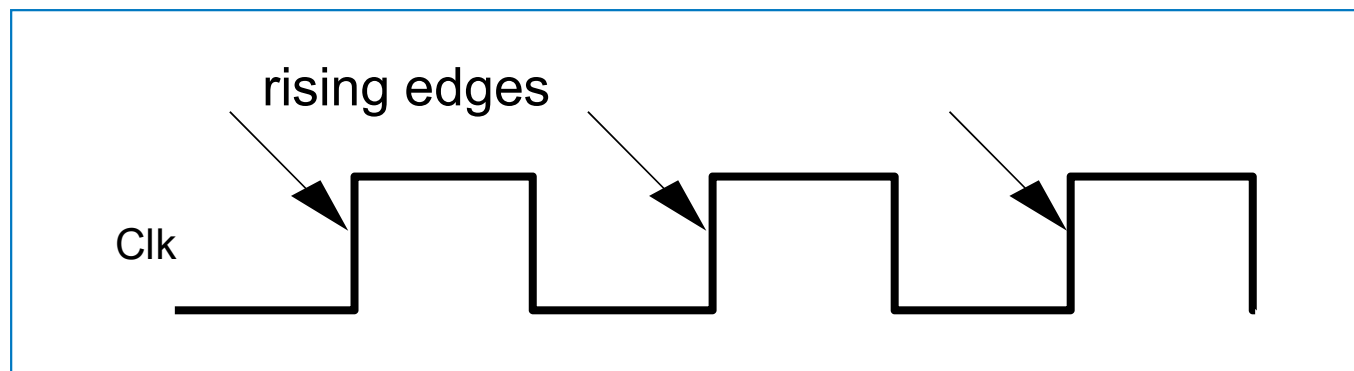
Is there any solution to
this ?

Problem with Level-Sensitive D Latch

- We want do the work: one per clock cycle
 - Independent of length of clock (1 time)



- Can we design bit storage that only stores a value on the rising edge of a clock signal?
 - There is exactly one rising edge per clock cycle
 - There is exactly one falling edge per clock cycle



Make Edge Sensitive Bit Storage

- Latch : Level sensitive storage
- Flip-Flop : Edge sensitive storage
 - Value get changed only at edges of clock



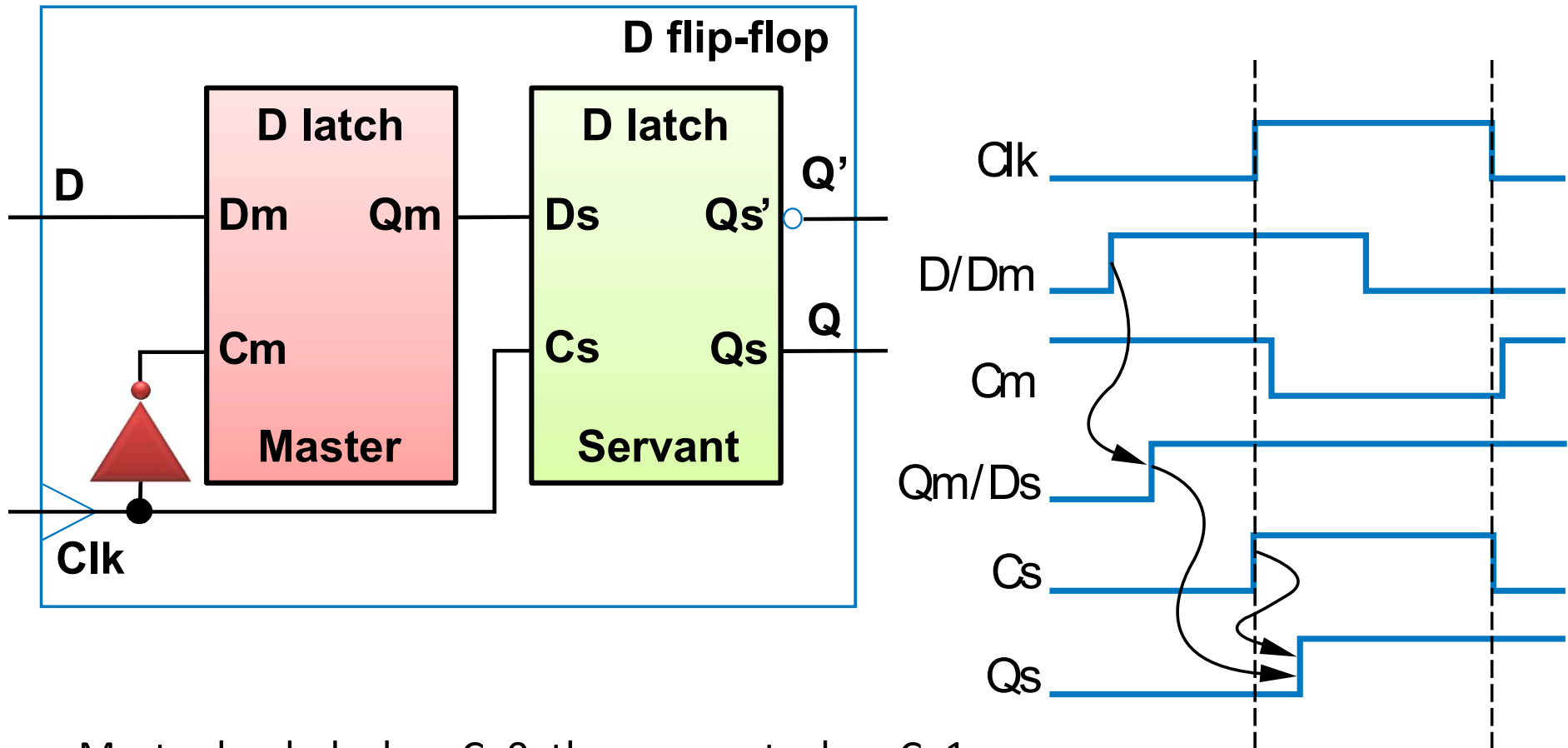
How to make a
Flip Flop out of
Latch?

Master -Slave D Flip-Flop

- **Two latches**, output of first goes to input of second, master latch has inverted clock signal
- So master loaded when $C=0$, then servant when $C=1$
- When C changes from 0 to 1, master disabled, servant loaded with value that was at D just before C changed
 - i.e., Value at D during rising edge of C

Master -Slave D Flip-Flop

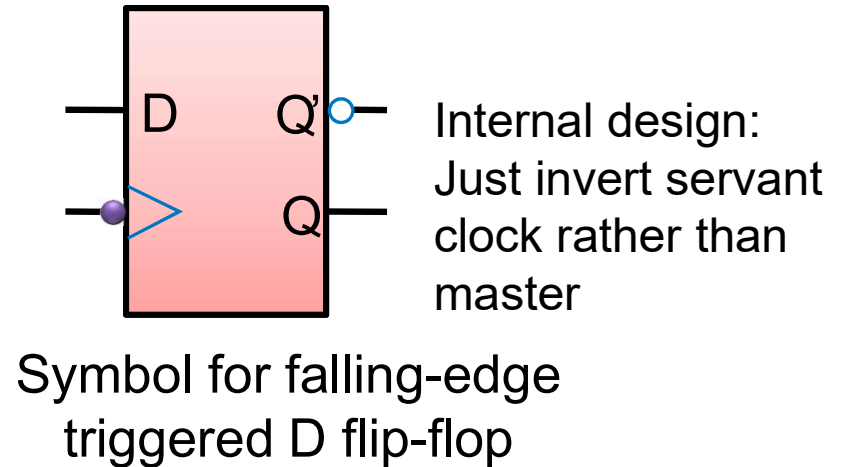
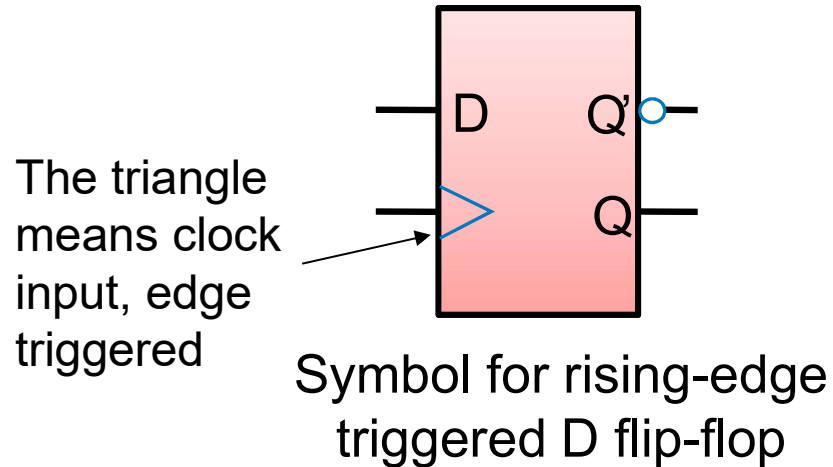
- **Flip-flop**: stores on clock edge, not level



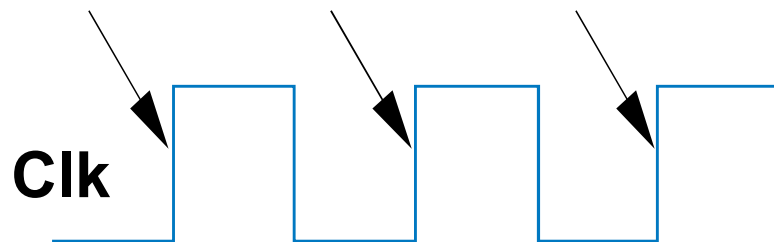
Master loaded when $C=0$, then servant when $C=1$

When C changes from 0 to 1, master disabled, servant loaded with value that was at D just before C changed -- i.e., value at D during rising edge of C

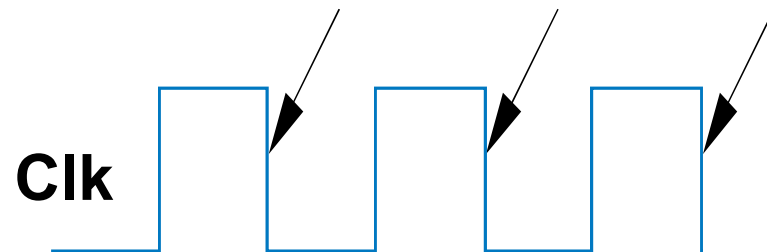
D Flip-Flop (Rising & Falling Edges)



Rising edges

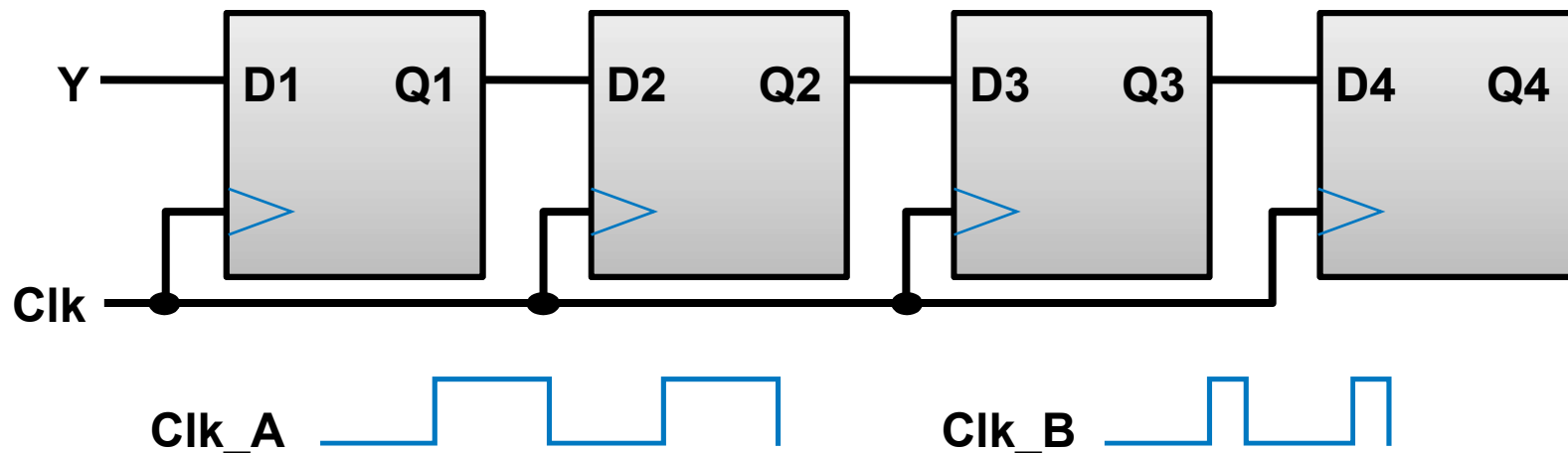


Falling edges



With D Flip-Flops (Solved)

- Solves problem of not knowing through how many latches a signal travels when $C=1$
 - Signal travels through exactly one FF, for Clk_A or Clk_B. **Why?**
 - **Because on rising edge of Clk, all four flip-flops are loaded simultaneously -- then all four no longer pay attention to their input, until the next rising edge. Doesn't matter how long Clk is 1.**

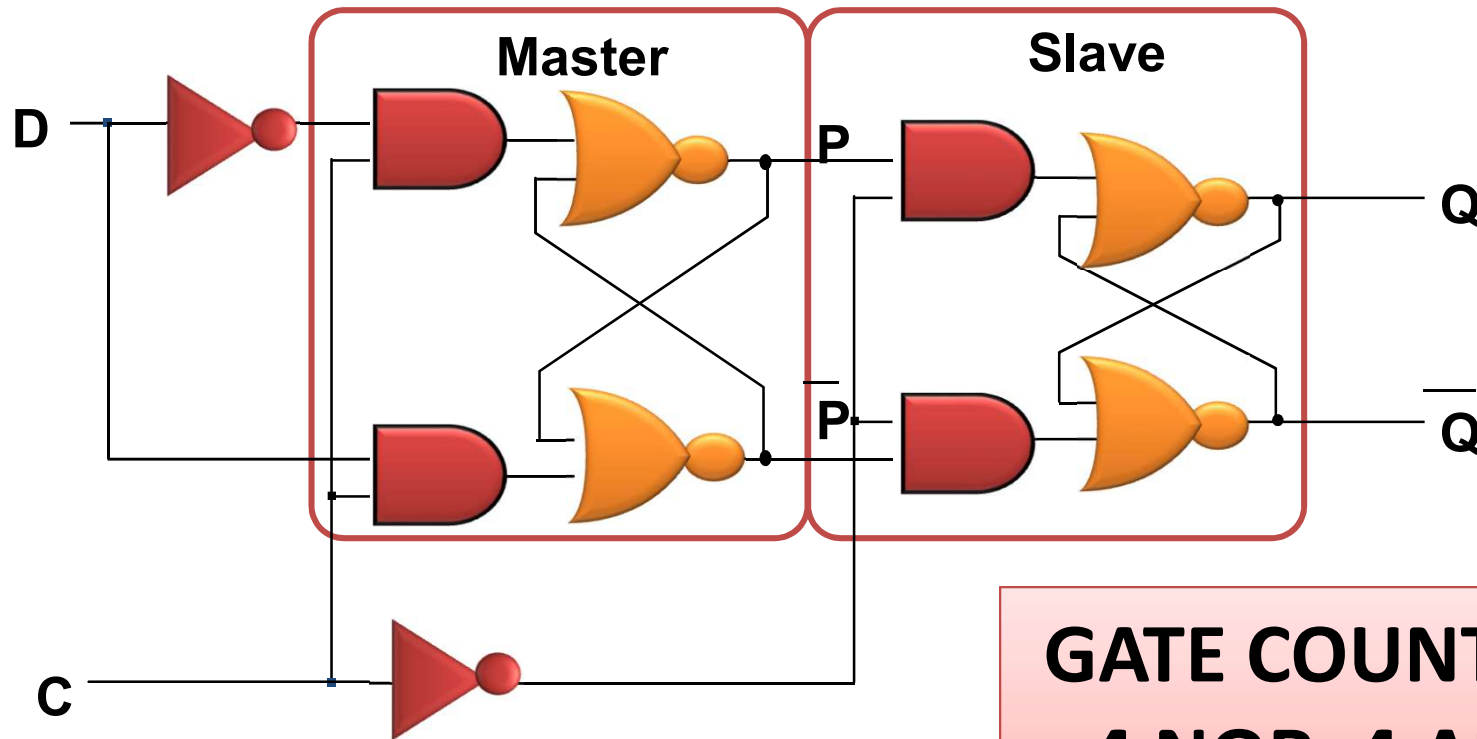


Two latches inside each flip-flop

D Latch vs. D Flip-Flop

- **Latch is level-sensitive:** Stores D when $C=1$
- **Flip-flop is edge triggered:** Stores D when C changes from 0 to 1
 - Saying “level-sensitive latch,” or “edge-triggered flip-flop,” is redundant
 - Two types of flip-flops -- rising or falling edge triggered.

Positive Edge Triggered D-Flip Flop: Optimization



GATE COUNT: 10
4 NOR, 4 AND
and 2 NOT

Verilog Code for MS-FF

```
module d_latch (en,d,q);  
input en, d;  
output reg q;  
always@(*)  
  begin  
    if(en) begin q=d; end  
end  
endmodule
```

Code for D-
Latch

```
module d_ff_struct (clk,d,q,qb);  
  
input clk,d;  
output q,qb;  
wire w1;  
  
  d_latch MASTER (~clk,d,w1);  
  d_latch SLAVE (clk,w1,q);  
assign qb=~q;  
  
endmodule
```

Code for D-FF with
M-S Latches

Verilog Code for MS-FF

```
module dff_test;  
reg D, iCLK;  
wire Q, Qb;  
integer i;  
d_ff_struct df(iCLK, D, Q, Qb);  
  
//Part A : Red part  
//Part B : Orange Part  
  
endmodule
```

```
initial begin  
iCLK=0;  
  for(i=0;i<10; i++) begin  
    #10; iCLK = ~iCLK;  
    $display("iCLK = %b, D = %b,  
      Q = %b, Qb = %b", iCLK, D, Q, Qb);  
  end  
end
```

```
initial begin  
$dumpfile("d_ff_str.vcd");  
$dumpvars;  
D = 0;  
#25; D = 1;  
#30; D = 0;  
#25; D = 1;  
end
```

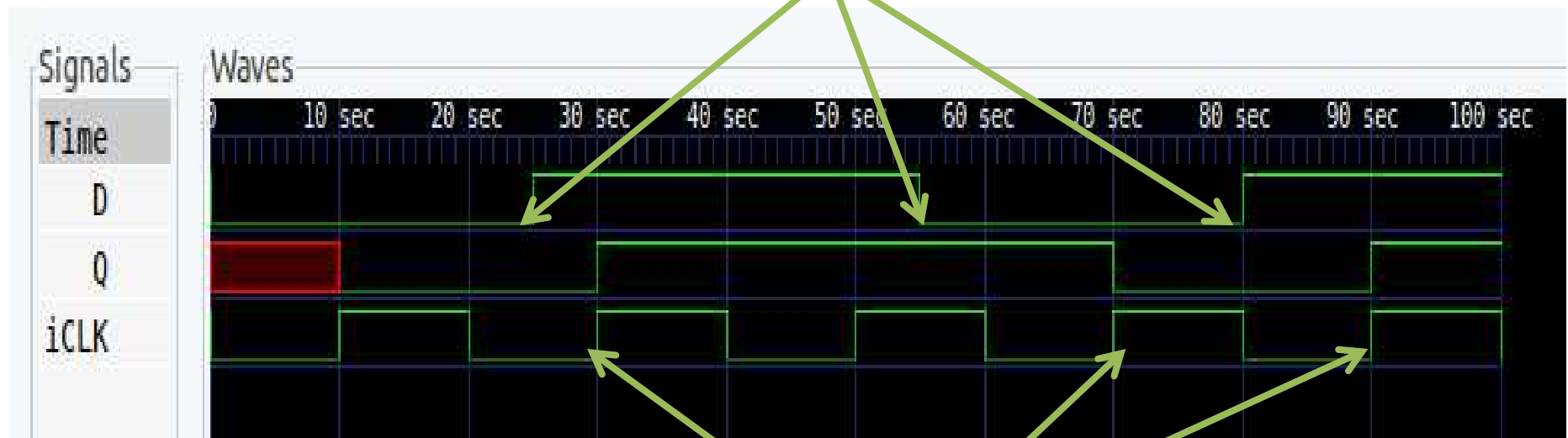
Waveform Simulation: MS-FF

\$iverilog d_ms_ff.v

\$/a.out

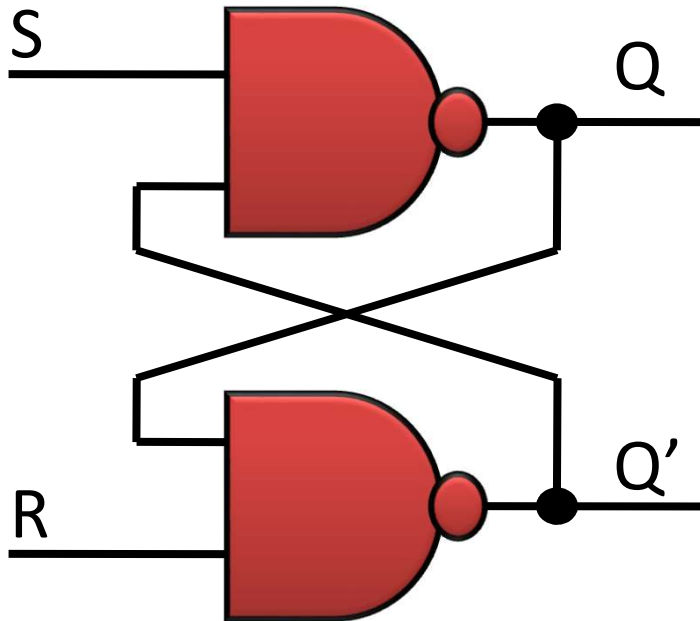
\$gtkwave d_ff_str.vcd

D value changed at



Q values reflected at positive clock edges

Remember: SR Latch with NAND Gates



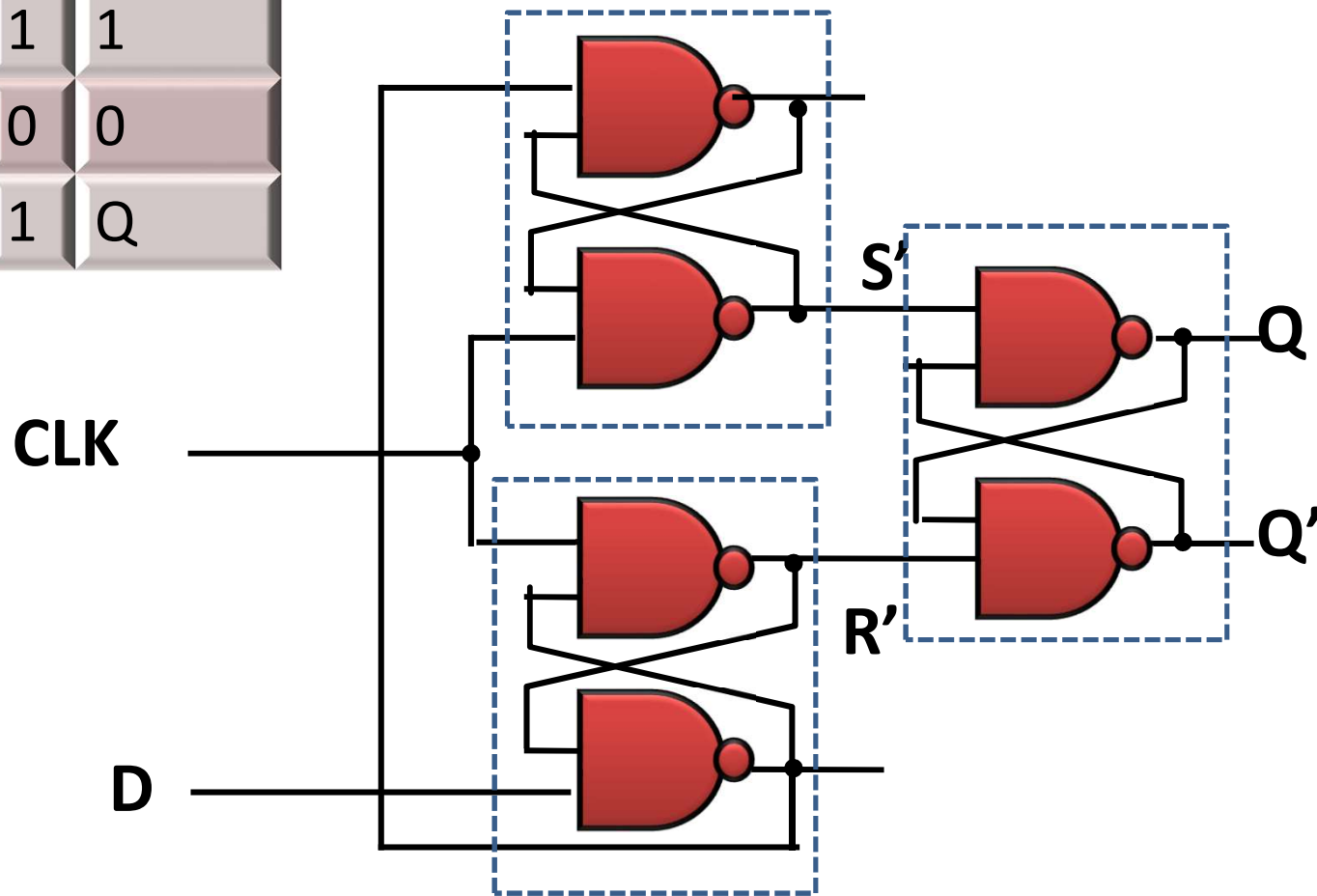
S	R	Q+
0	0	1*0* (Unpredictable)
0	1	1
1	0	0
1	1	Q

Opposite to SR Latch with NOR Gates

Set will do $Q=0$ and Reset will $Q=1$

Positive-Edge Triggered D-FF: Economical

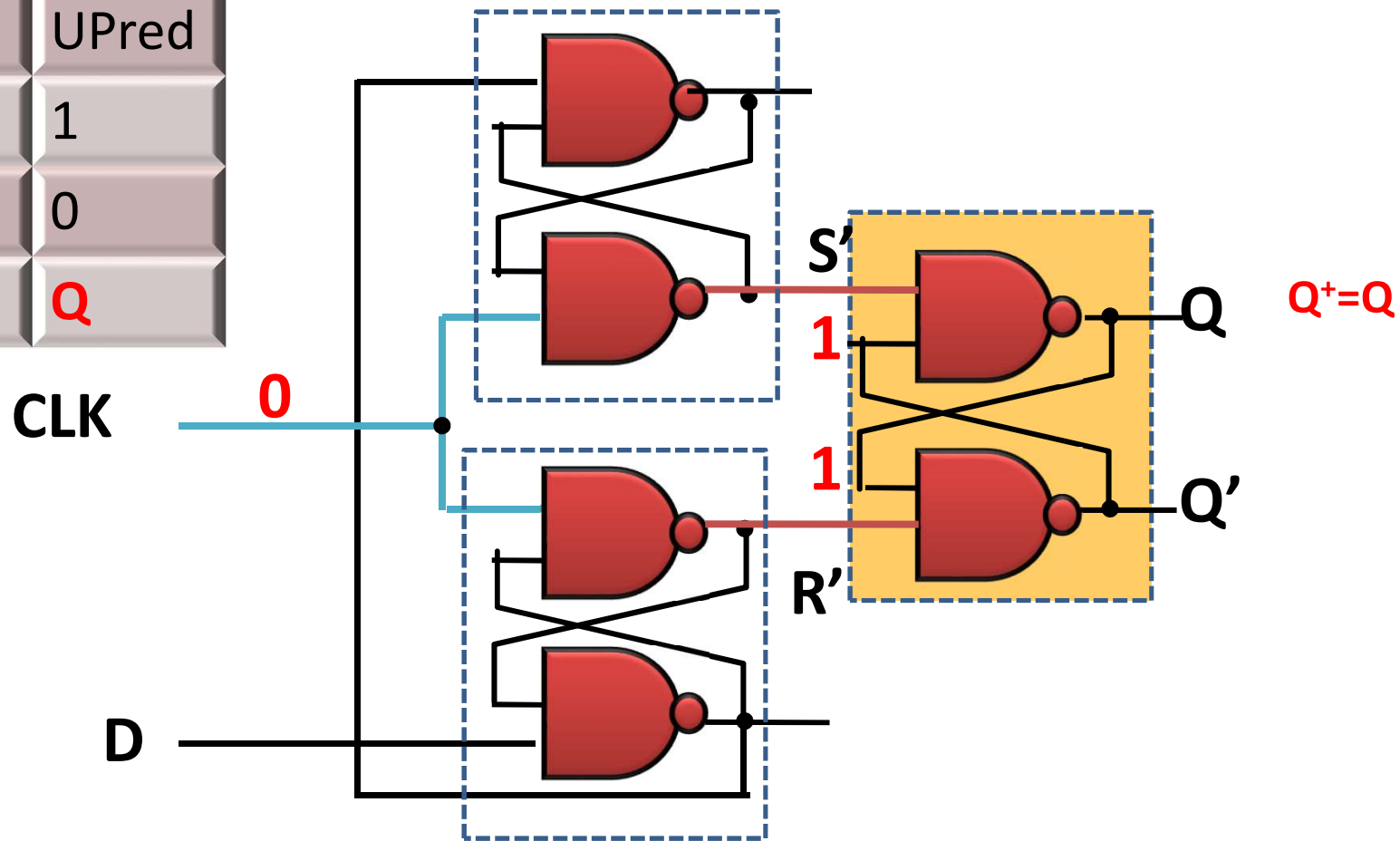
S	R	Q+
0	0	UPred
0	1	1
1	0	0
1	1	Q



Positive-Edge Triggered D-FF: Economical

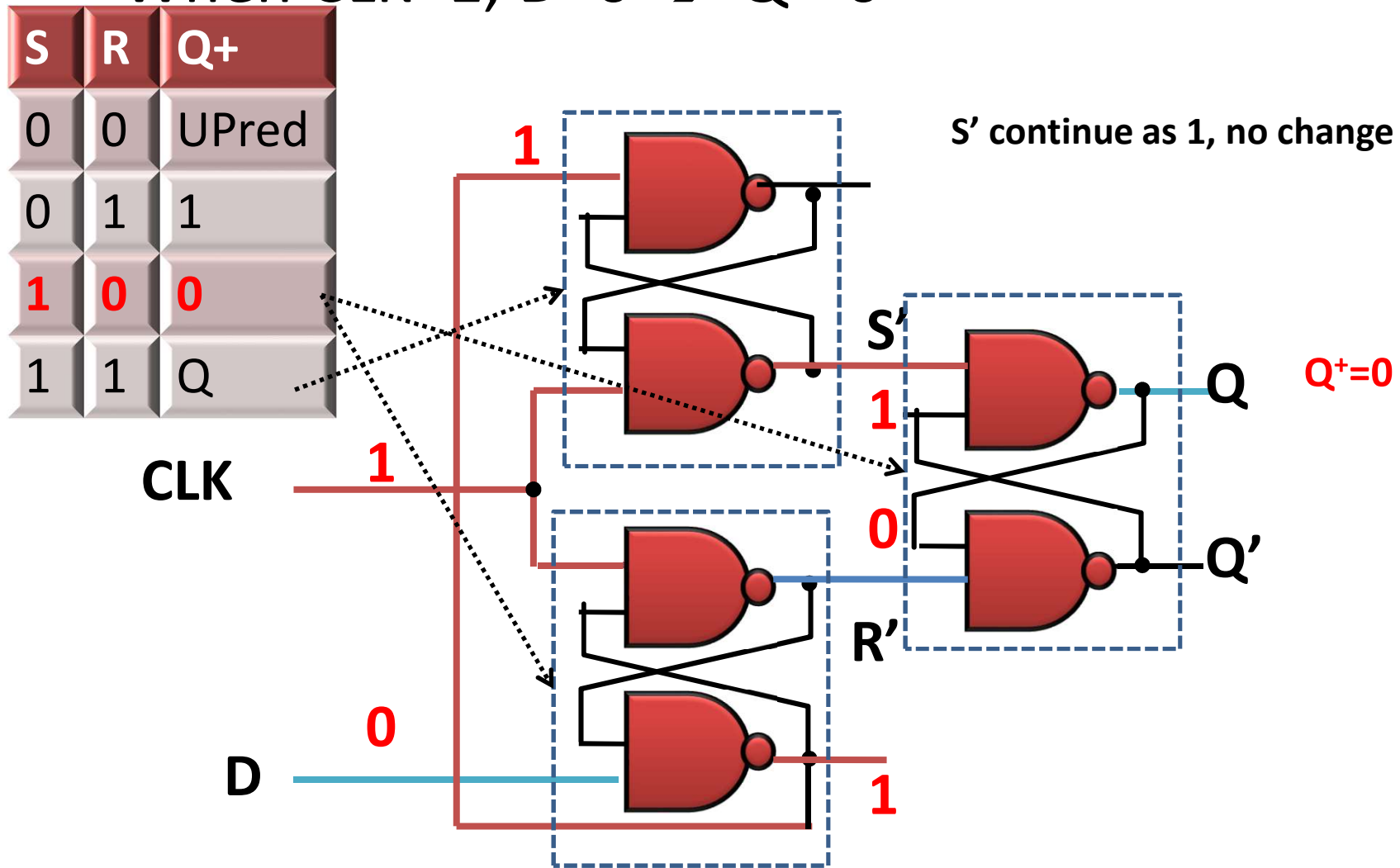
- When $\text{CLK}=0$, $S'R'=11$, $Q^+=Q$ (Independent of D)

S	R	Q^+
0	0	UPred
0	1	1
1	0	0
1	1	Q



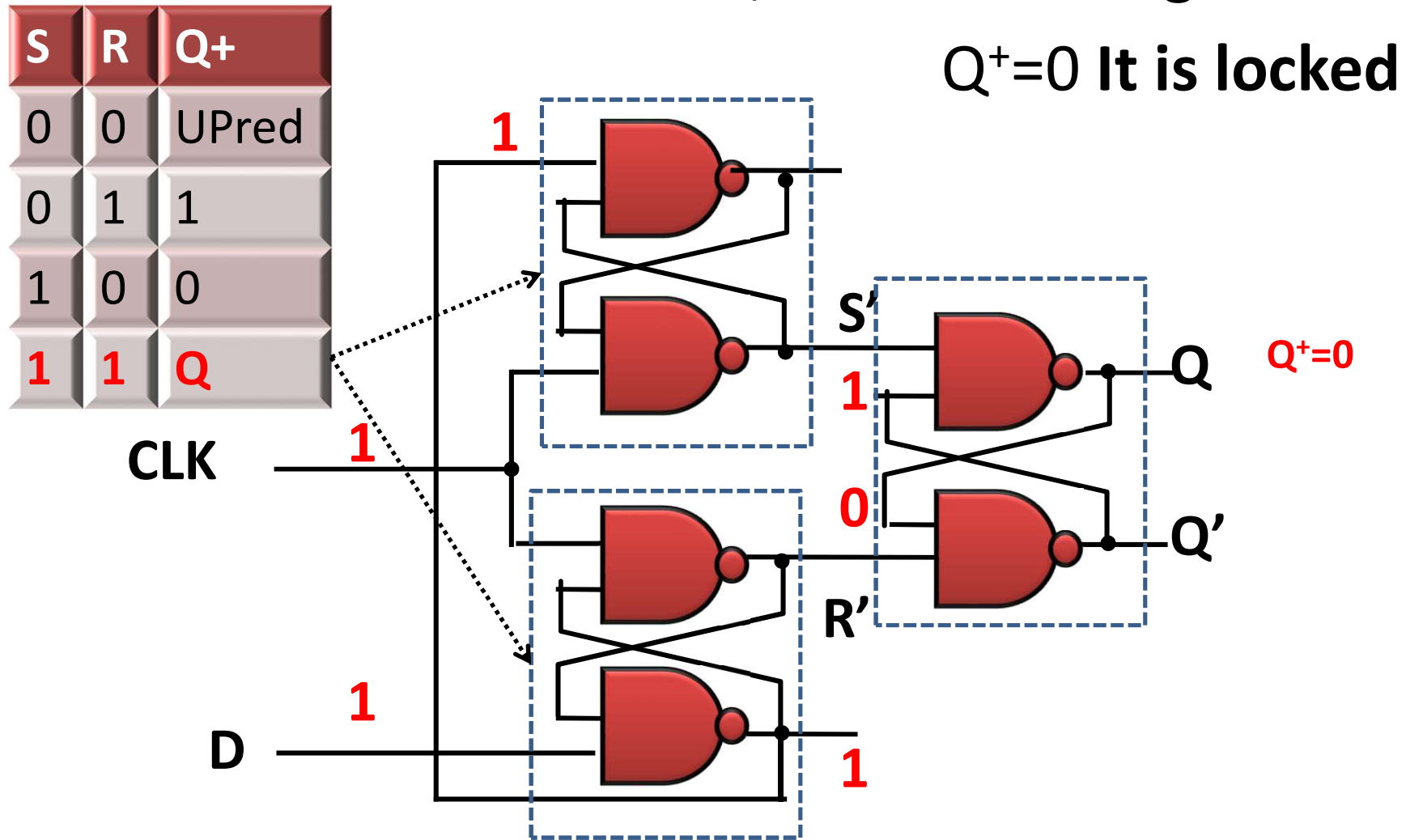
Positive-Edge Triggered D-FF: Economical

- When $CLK=1$, $D=0 \Rightarrow Q^+=0$



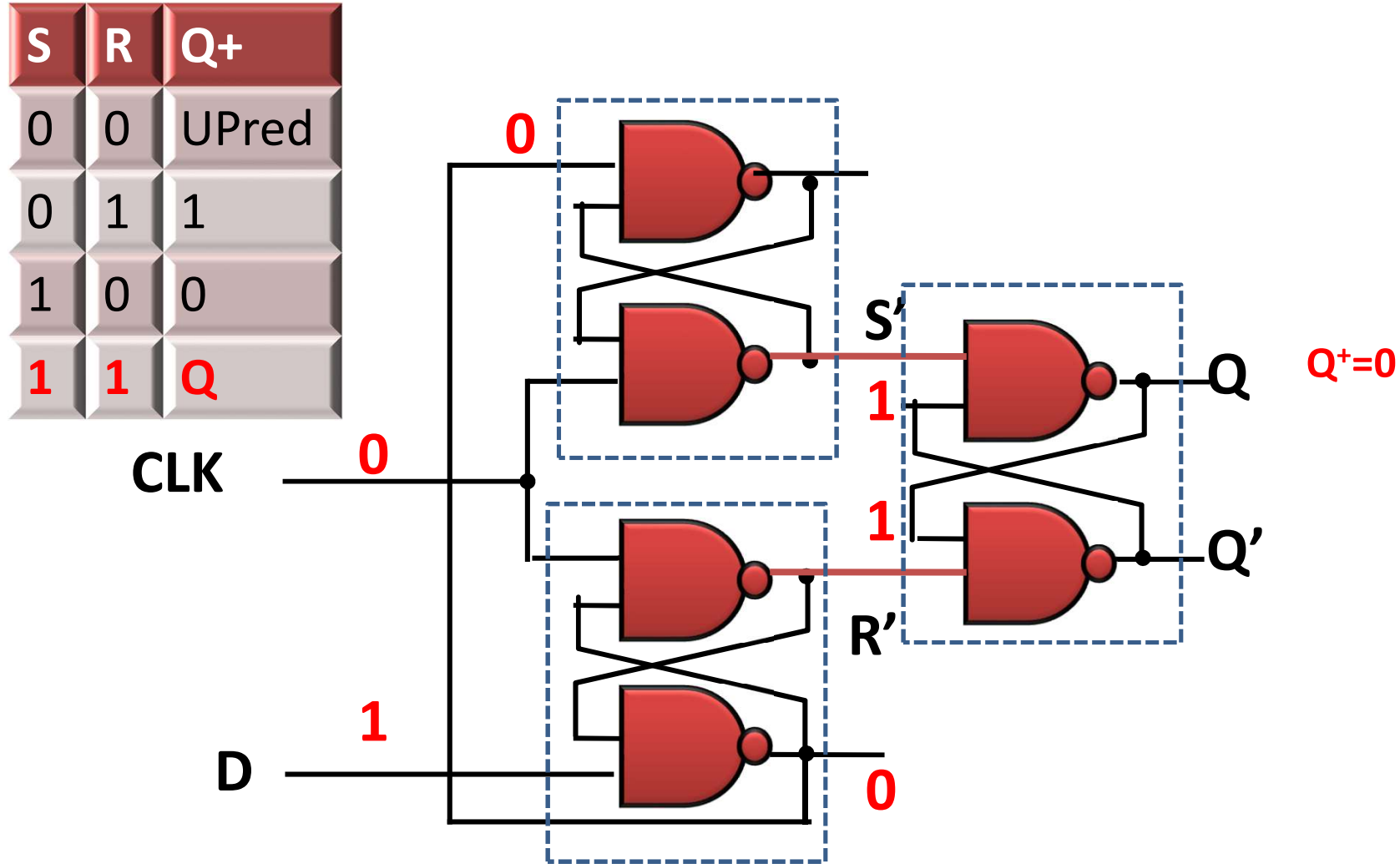
Positive-Edge Triggered D-FF: Economical

- After that When CLK=1, D=1: No changes to R'S':



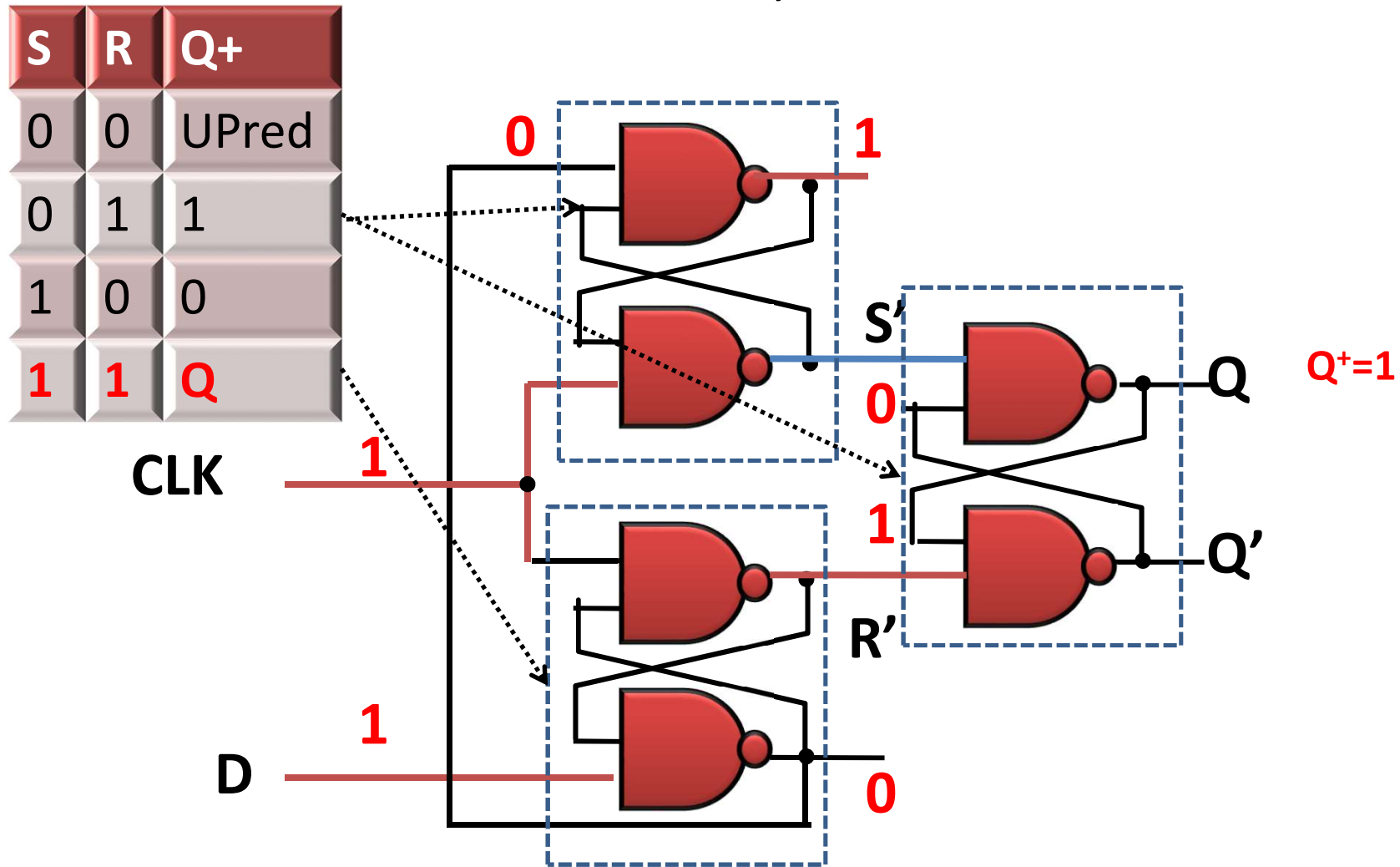
Positive-Edge Triggered D-FF: Economical

- After that When $\text{CLK}=0$, $D=1$: $Q^+=Q$ (Indt. of D)

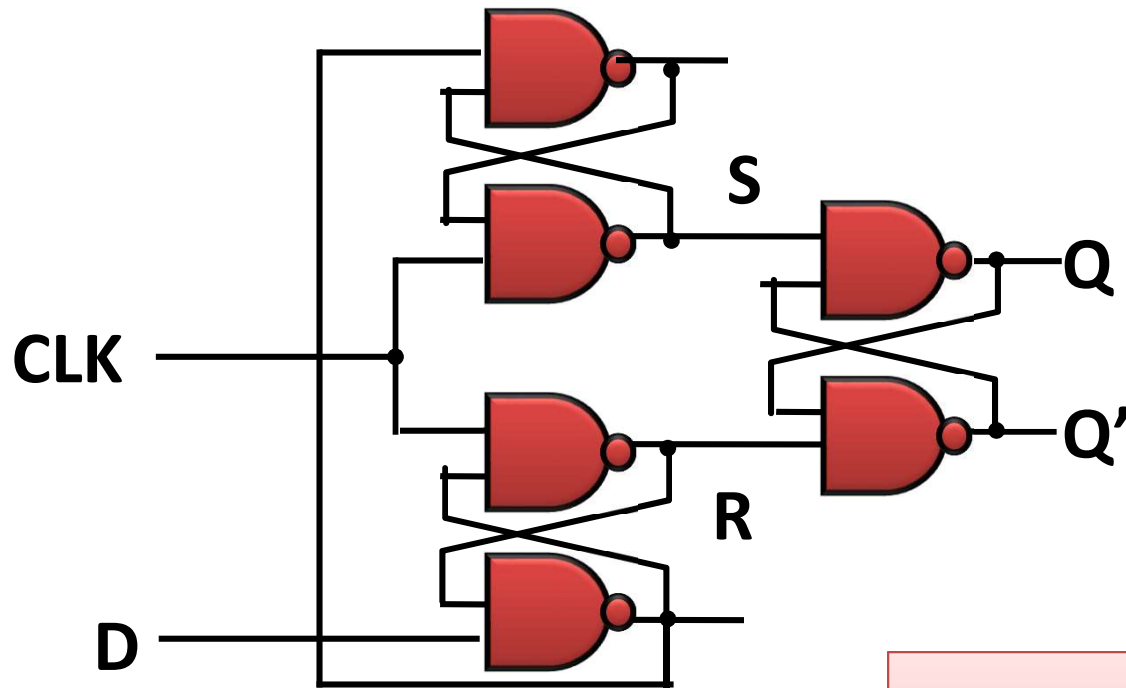


Positive-Edge Triggered D-FF: Economical

- After that When CLK=1, D=1: $\Rightarrow Q^+ = 1$



Positive-Edge Triggered D-FF: Economical



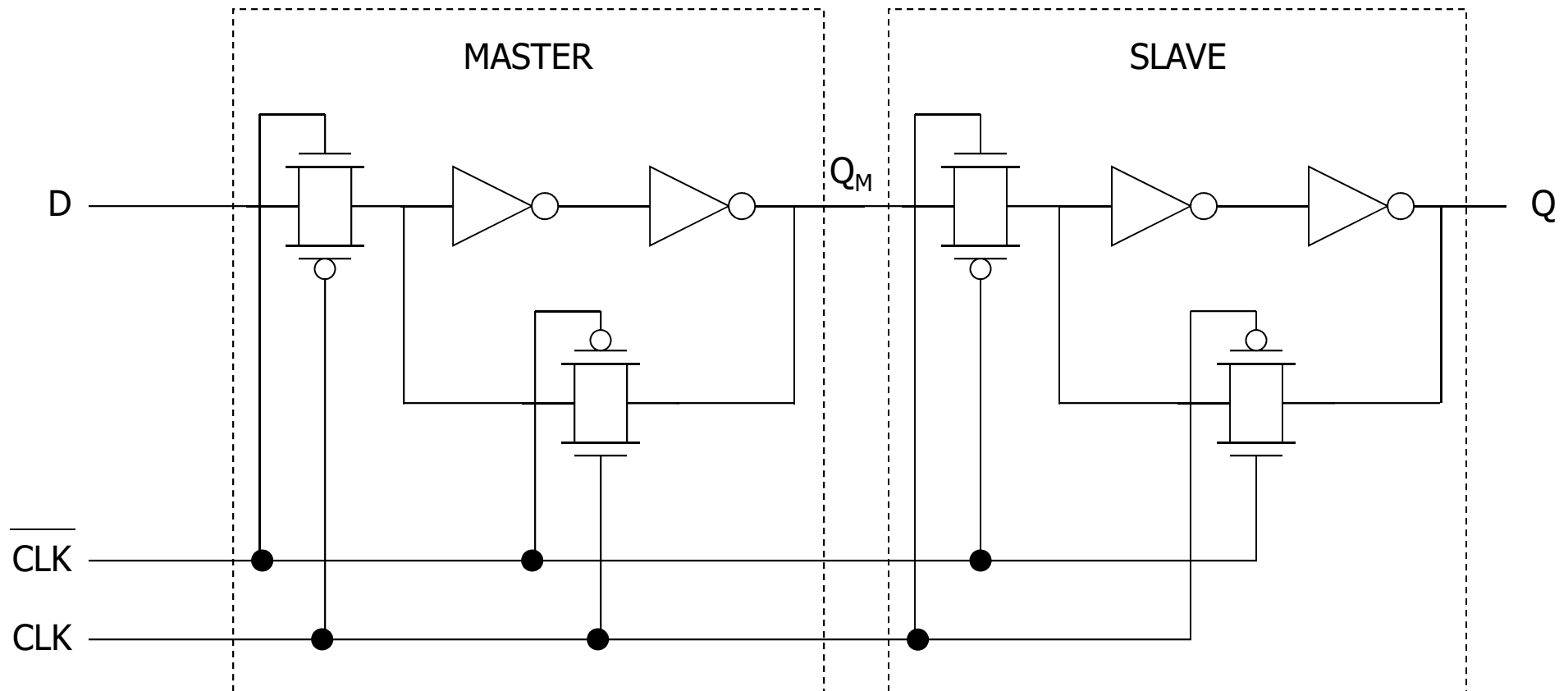
GATE COUNT: 6
6 NAND, Same types

**Transistor level optimization
is
out of syllabus**

But showing two slides 😊 😊 😊

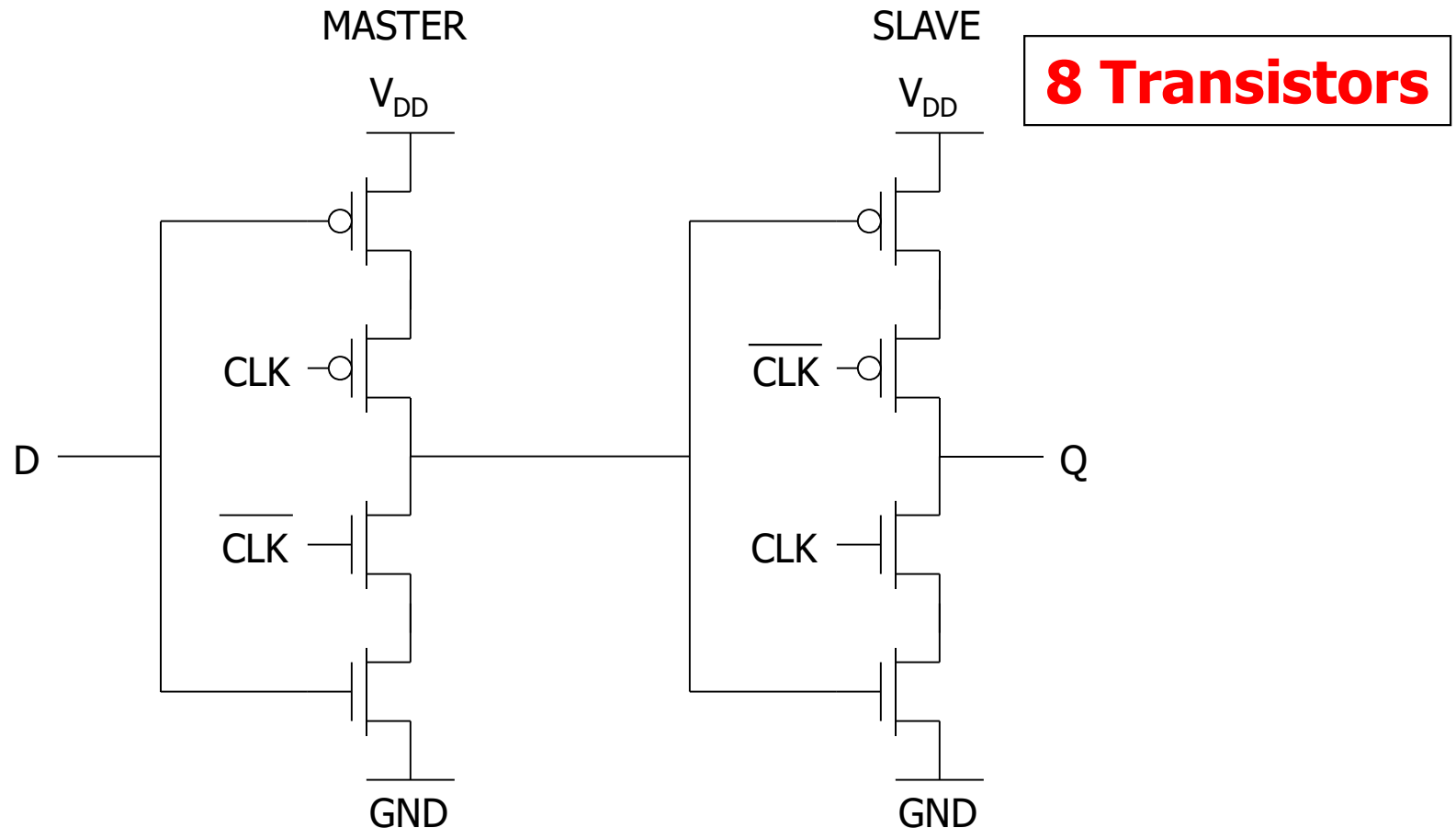
Master-Slave Edge-Triggered Flip-Flop

2 x 8 = 16 Transistors



More Efficient Master-Slave Edge-Triggered Flip-Flop

- Called a C²MOS (Clocked CMOS) design



Problem handled in Designing FF

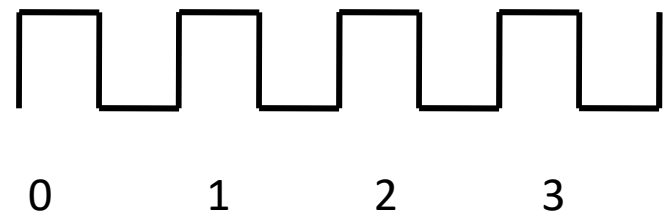
- OR Gate : worked just like a ringing bell
- OR gate with Feed back : ($Q=1$ can never be changed)
- Two NOR gates with cross coupled out put and input : Solved to store a bit but Race condition
- Ensure $RS=11$ will not happed by adding Not and AND gate
- Enable Signal to put remove : delay of added Ckt
- Master Slave Latches to make a FF
- Optimized D-FF (using only 6 NAND Gates)
- **Transistors and CMOS level Optimization (8 CMOS transistor)**

Conventions

- The circuit is *set* means output = 1
- The circuit is *reset* means output = 0
- Flip-flops have two output Q and Q'
- Due to time related characteristic of the flip-flop:

– Q_t or Q: present state

– Q_{t+1} or Q^+ : next state

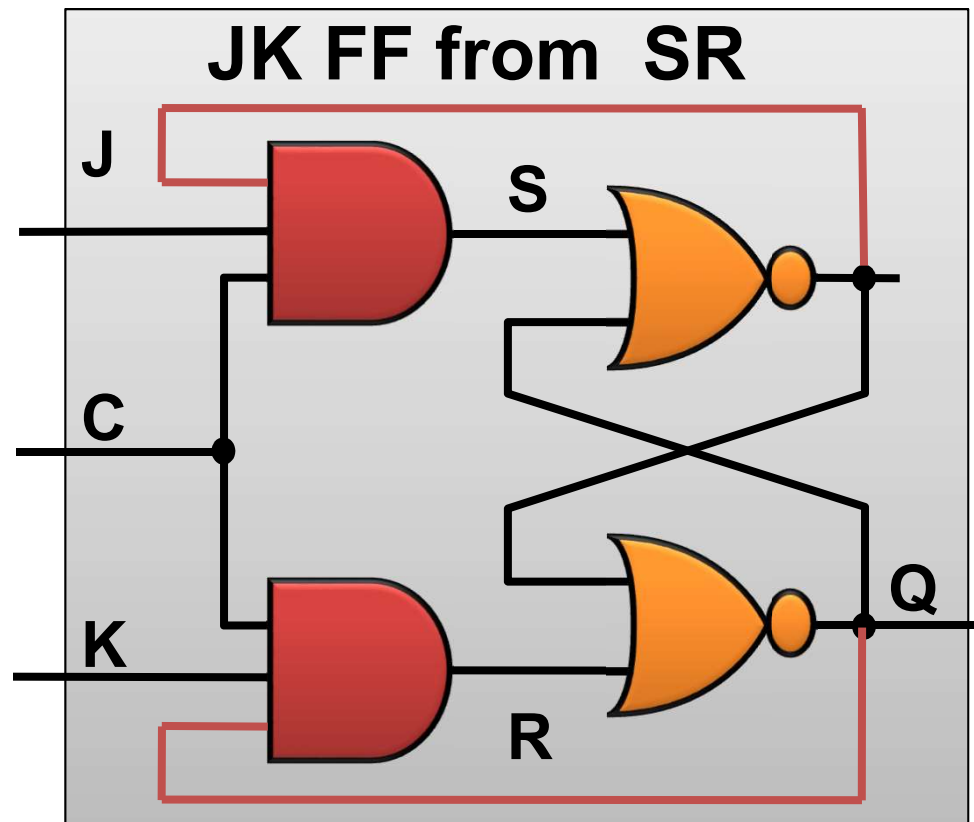


4 Type of Flip Flop

- **SR Flip Flop** : Set/Reset Flip Flop
- **D Flip Flop** : Data Flip Flop to store Bit
- **J-K Flip Flop**: Unavoidable SR=11 state to Toggle (All input values are useful)
 - The JK Flip Flop was named to honour "Jack Kilby" of Texas Instrument engineer who invented the concept of IC.
- **T Flip Flop**: Toggle Flip Flop

J-K Latches

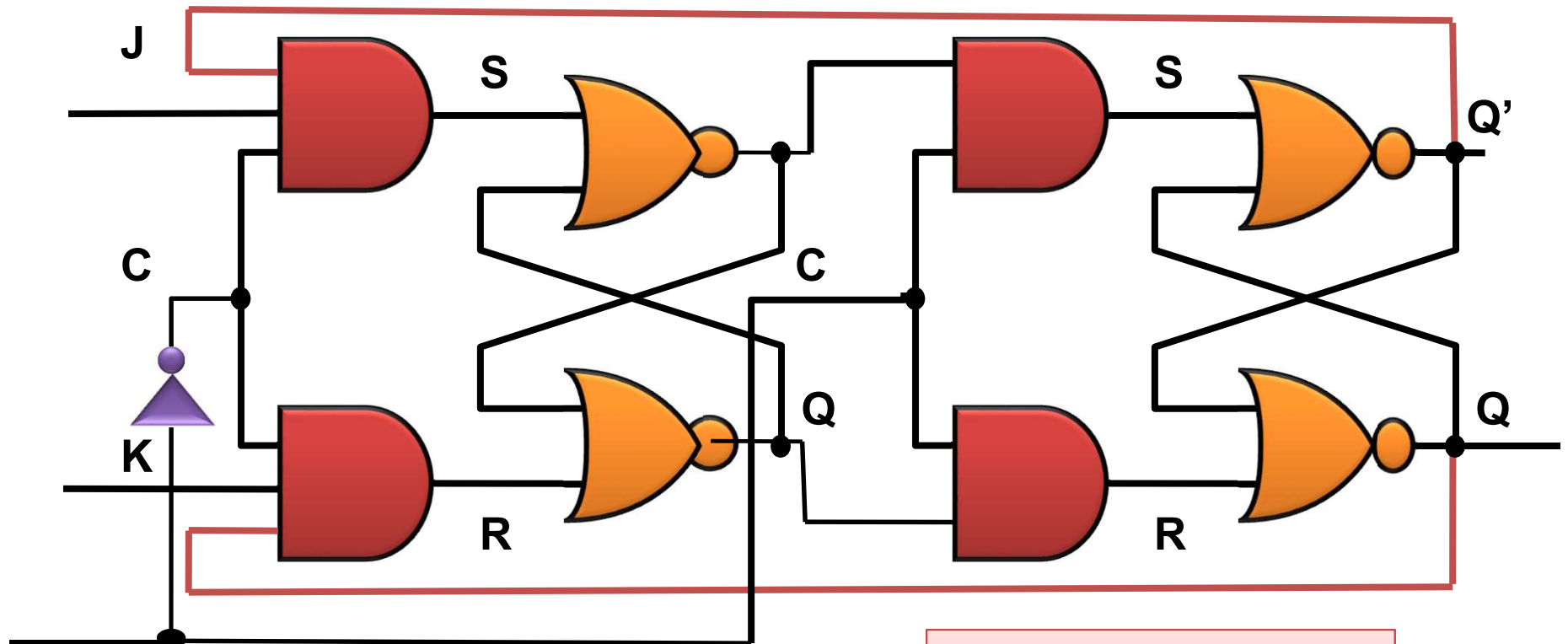
- The JK flip-flop augments the behavior of the SR flip-flop (J=Set, K=Reset) by interpreting the $S = R = 1$ condition as a "flip" or toggle command.



J	K	Q+
0	0	Qt
0	1	0
1	0	1
1	1	Qt'

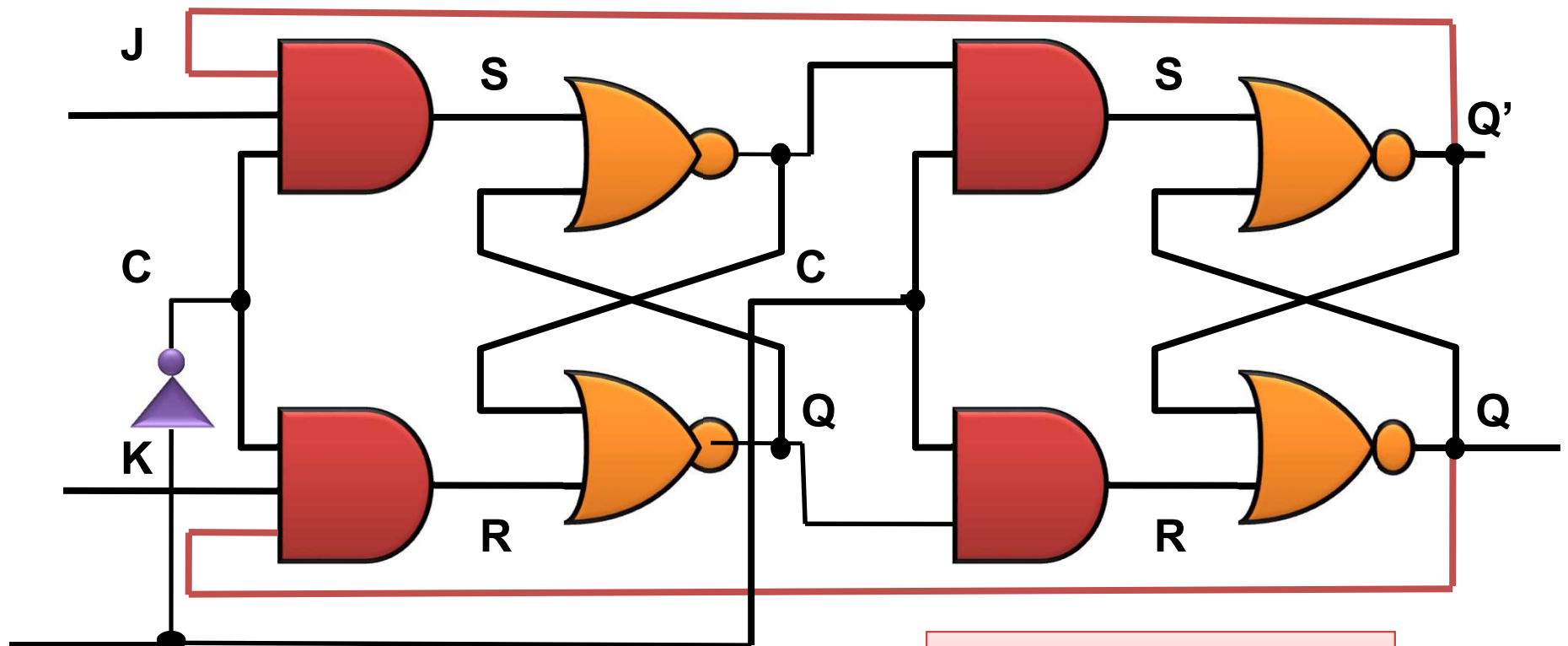
$$Q^+ = K'Q + JQ'$$

Master Slave J-K Flip Flop



$$Q^+ = K'Q + JQ'$$

Master Slave J-K Flip Flop



$$Q^+ = K'Q + JQ'$$

J-K Flip Flop

- To synthesize a D flip-flop, simply set K equal to the complement of J.
- The JK flip-flop is a universal flip-flop
 - Because it can be configured to work as any FF
 - T flip-flop or D flip-flop or SR flip-flop.

J=T	K=T	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '

J=D	K=D'	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '

J=S	K=R	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '

Verilog Code for JK-FF

```
module jk_ff(input j, input k, input clk, output q);  
  reg q;  
  always @ (posedge clk)  
    case ({j,k})  
      2'b00 : q <= q;  
      2'b01 : q <= 0;  
      2'b10 : q <= 1;  
      2'b11 : q <= ~q;  
    endcase  
endmodule
```

Code for JK-FF

Verilog Code for JK-FF

```
module jk_ff_test;  
reg J K , iCLK;  
wire Q ;  
integer i;  
jj_ff df(J, K, iCLK, Q);
```

```
//Part A : Red part  
//Part B : Orange Part
```

```
endmodule
```

```
initial begin
```

```
iCLK=0;
```

```
for(i=0;i<10; i++) begin
```

```
#10; iCLK = ~iCLK;
```

```
$display("iCLK = %b, J = %b,  
K = %b, Q = %b", iCLK, J, K, Q);
```

```
end
```

```
end
```

```
initial begin
```

```
$dumpfile("jk_ff.vcd");
```

```
$dumpvars;
```

```
J <= 0; K <= 0;
```

```
#5 J <= 0; K <= 1;
```

```
#20 J <= 1; K <= 0;
```

```
#20 J <= 1; K <= 1;
```

```
#20 $finish;
```

```
end
```

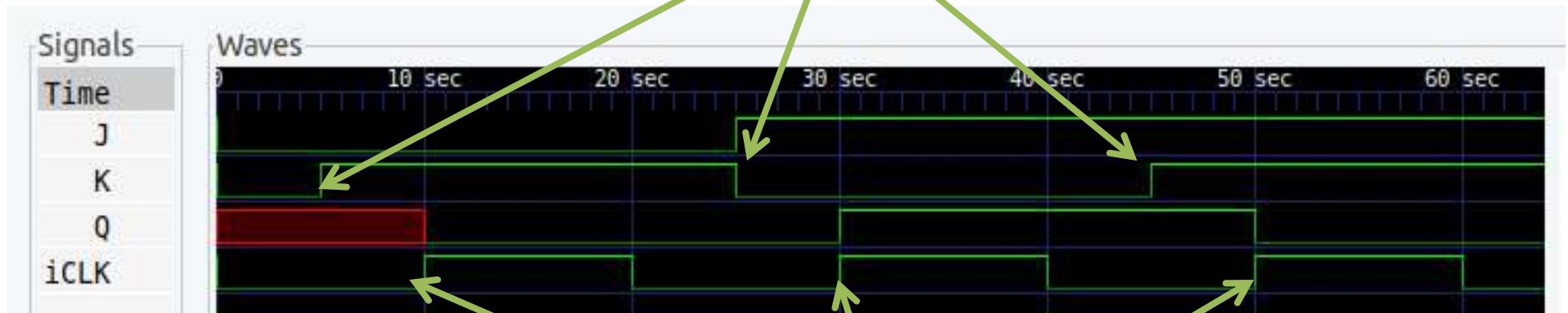
Verilog Code for JK-FF

\$iverilog jk_ff_behav.v

\$/a.out

\$gtkwave jk_ff.vcd

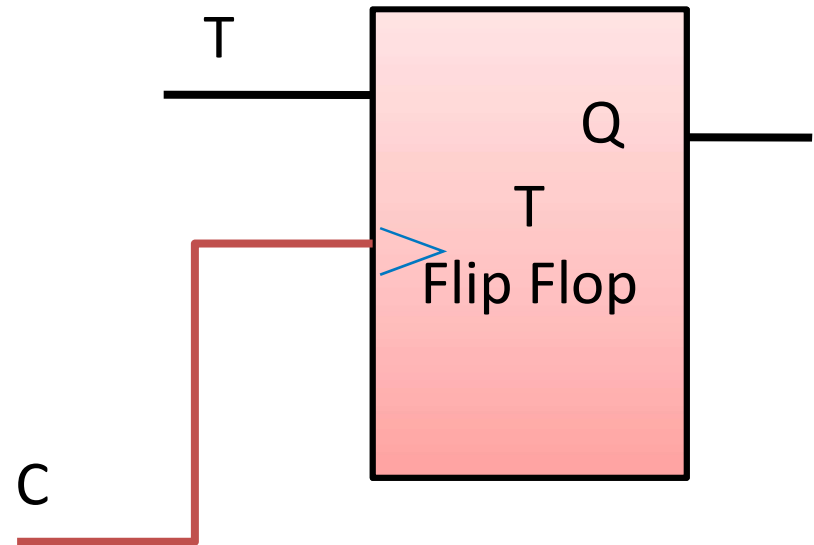
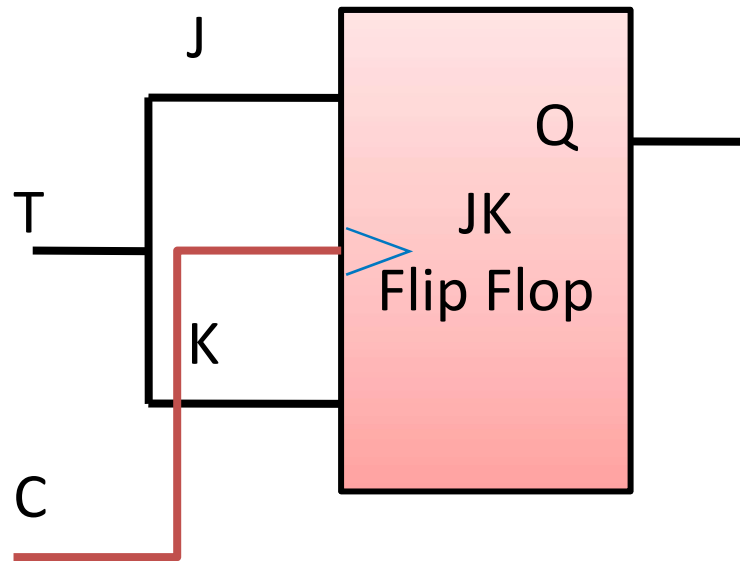
J or K value changed at



Q values reflected at positive clock edges

Toggle Flip-Flop: T FF

- $J=K=1$, $Q^+ = Q'$



Verilog Code for T-FF

```
module t_ff(input t, input clk, output q);  
  reg q;  
  always @ (posedge clk)  
  begin  
    q<=(t ? ~q : q);  
  end  
endmodule
```

Code for T-FF

4 Types of Flip-Flops

S	R	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	U



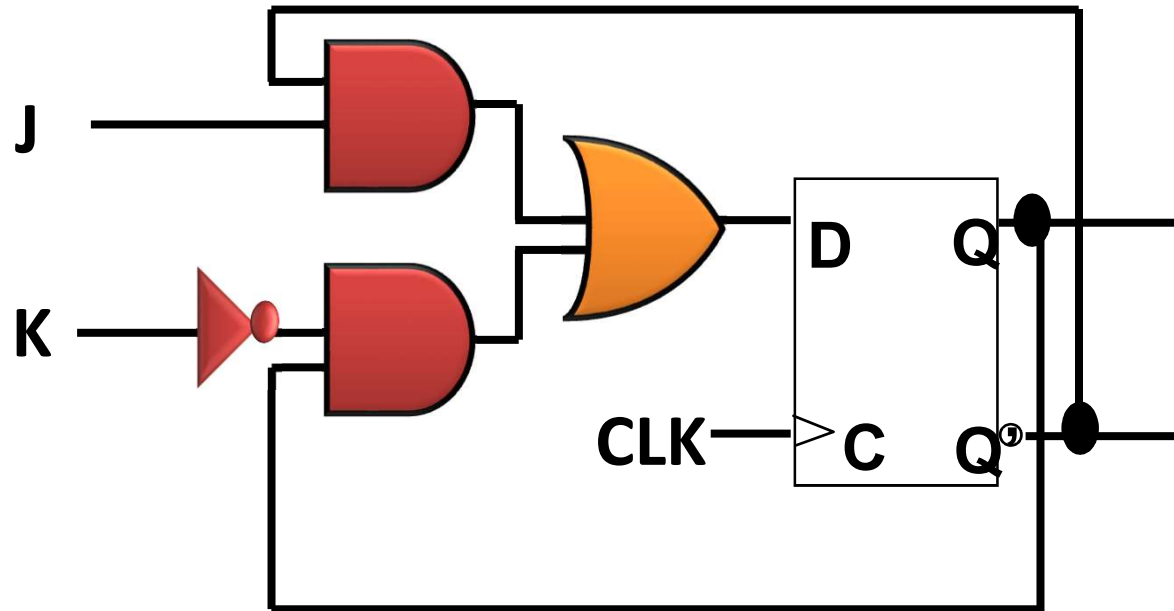
D	Q+
0	0
1	1

J	K	Q+
0	0	Q _t
0	1	0
1	0	1
1	1	Q _t '



T	Q+
0	Q _t
1	Q _t '

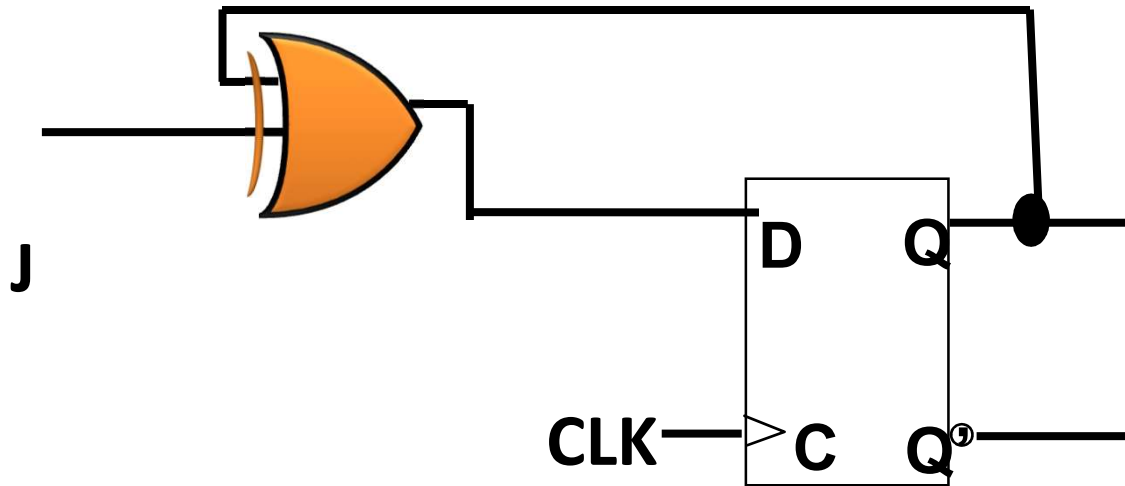
Given a D FF: Construct JK FF



J	K	D	$K'Q + JQ'$
0	0	Q	$1Q + 0Q'$
0	1	0	$0Q + 0Q'$
1	0	1	$1Q + 1Q'$
1	1	Q'	$0Q + 1Q'$

$$D = K'Q + JQ'$$

Given a D FF: Construct T FF



T	D	$TQ' + T'Q$
0	Q	$0Q' + 1Q$
1	Q'	$1Q' + 0Q$

$$D = T Q' + T' Q$$

Characteristic Equations

- A descriptions of the next-state table of a flip-flop
- Constructing from the Karnaugh's map for Q_{t+1} in terms of the present state and input

Characteristic tables

- The tables that we've made so far are called **characteristic tables**.
 - They show the next state $Q(t+1)$ in terms of the current state $Q(t)$ and the inputs.
 - For simplicity, the control input C is not usually listed.
 - Again, these tables don't indicate the positive edge-triggered behavior of the flip-flops that we'll be using.

J	K	Q^+
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

D	Q^+
0	0
1	1

T	Q^+
0	Q_t
1	Q_t'

Characteristic equations

- We can also write **characteristic equations**, where the next state $Q(t+1)$ is defined in terms of the current state $Q(t)$ and inputs.

J	K	Q+
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

$$\begin{aligned}
 Q^+ &= J'K'Q + JKQ_t' + JK' \\
 &= J'K'Q + JKQ' + JK'Q' + JK'Q \\
 &= J'K'Q + JK'Q + JKQ' + JK'Q' \\
 &= (J+J')K'Q + J(K+K')Q'
 \end{aligned}$$

$$Q^+ = K'Q + JQ'$$

$$Q(t+1) = K'Q(t) + JQ'(t)$$

Characteristic equations

- We can also write **characteristic equations**, where the next state $Q(t+1)$ is defined in terms of the current state $Q(t)$ and inputs.

D	Q^+
0	0
1	1

$$Q^+ = D$$

$$Q(t+1) = D$$

T	Q^+
0	Q_t
1	Q_t'

$$Q^+ = T'Q + TQ' = T \oplus Q$$

$$Q(t+1) = T'Q(t) + TQ'(t) = T \oplus Q(t)$$

Characteristic equations

	SR			
	00	01	11	10
0	0	0	-	1
1	1	0	-	1

$$Q^+ = S + R'Q$$

Flip Flop Type	Characteristic Equation
SR	$Q^+ = S + R'Q$
JK	$Q^+ = JQ' + K'Q$
D	$Q^+ = D$
T	$Q^+ = TQ' + T'Q = T \oplus Q$

FF with Asynchronous Inputs

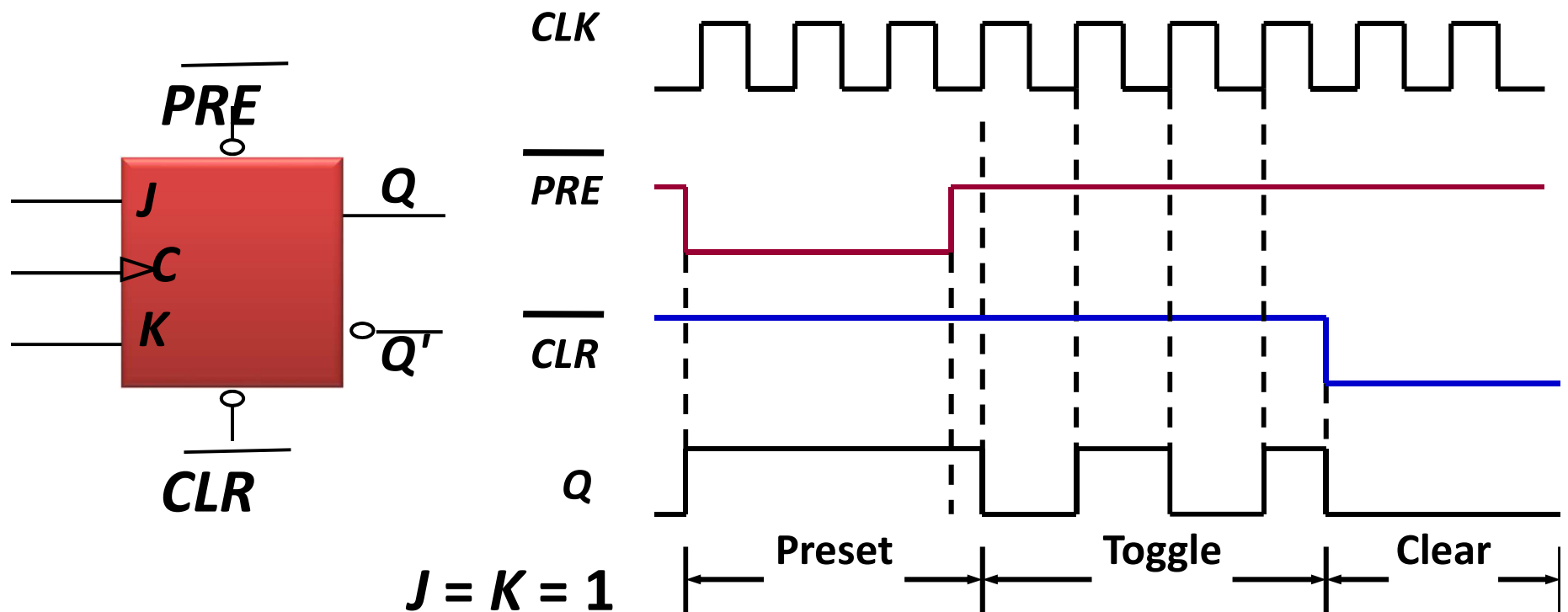
- S-R, D and J-K inputs are synchronous inputs
 - As data on these inputs are transferred to the flip-flop's output
 - Only on the triggered edge of the clock pulse.
- **Asynchronous** inputs affect the state of the flip-flop independent of the clock
- Example:
 - *Preset (PRE) and clear (CLR)*
 - or *direct set (SD) and direct reset (RD)*

FF with Asynchronous Inputs

- When $PRE=HIGH$
 - Q is immediately set to HIGH.
- When $CLR=HIGH$
 - Q is immediately cleared to LOW.
- When both PRE and CLR are LOW.
 - Flip-flop in normal operation mode

Asynchronous Inputs

A J-K flip-flop with active-LOW preset and clear inputs



Thanks